HLD (High Level Design)

* Topics:

1. Network Protocols — TCP, Websocket, HTTP etc.
2. Client - Server Vs Peer 2 Peer Arch.
3. CAP Theorem. *
4. Microservices Imp. Design Patterns
   - Decomposition Pattern
   - SAGA Pattern **
   - Strangler Pattern *
   - CORS
5. Scale from 0 to Million Users *
6. Design URL Shortening
7. Back of the Envelope Estimation *
8. Design Key-Value store
9. SQL v/s NoSQL, when to use whic DB
10. Design WhatsApp
11. Design Rate limitter
12. Design Search Autocomple System / Typeahead System
13. understand more HLD components:
    - Message Queue
    - Kafka
    - Proxy Servers.
    - Block storage, S3; RAID
    - CDN
    - file system
    - Bloom filter → Merkel Tree, Gossiping Protocol
14. Consistent Hashing Design *
15. Caching: caching invalidation, cache eviction
16. How to scale DB:
    - sharding (Horizontal & Vertical)
    - Replication, Mirroring
    - Leader Election
    - Indexing
17. Design Notification System .....

Network Protocols :- (7 layers)

→ we will read about Application layer.

Application

Client Server Protocols — Peer 2 Peer Protocols

CN notes — HTTP — 1 connection — webRTC

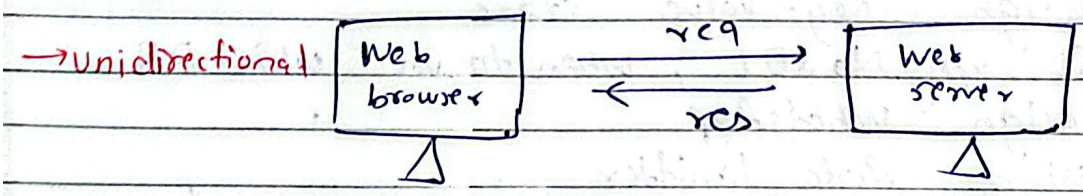
CN notes — FTP — 2 connection, (data & control)

— SMTP — IMAP or PoP

— Web Sockets

CN notes — TCP/IP

* Client - Server :



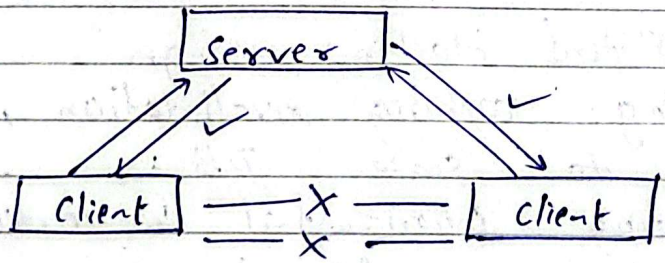
→ Web socket : bidirectional

↳ NOT Peer 2 Peer

client —can talk to→ Server

Server —talk to→ Client

why → as client can't talk to each other



web socket → chat system, as in simple unidirection there will be too many req.

- Client - Server arch is a sys. des. model where multiple clients req. services or resources from a centralized server.
- server manages data, processing, resources, while client provides the user interface and interact with server.

ex: • Banking System :: client allow users to check balance & & perform transactions. Server procces req. and securely manage customer acct data.
• Healthcare

• Imporatance in Sys des.

(1) centralized management : simplifies maintinance, monitoring and security by managing res. from a single server.

(2) Scalability : Support growth by adding or upgrading servers without major changes to the system.

(3) Resource optimt^n: Efficiently distributes workload b/w clients (interaction) & servers (processing & storage).

(4) Reliability and Availability: Ensures continuous service using redundancy, backups & load balancing.

(5) Enhanced security : bcz of centralized control, encryption & auth.

• Email Protocols:

→ A email protocol is a set of rules that define how emails are sent, received and stored b/w mail servers & clients.

→ 3 primary email protocols
(1) SMTP – Simple Mail Transfer Protocol
(2) IMAP – Internet Message Access Protocol
(3) POP3 – Post Office Protocol (3-version)

① SMTP Purpose: sending emails

Working
- Transfer email from sender → mail server → receiver ← recipent's mail server ←
- Uses DNS to find the ~~the~~ recipents's mails server.
- Only sends emails; receiving is handled by IMAP / POP3
- If delivery fails → bounce-back-error.
- Support email relaying and multiple recipients
  (relaying: from one server to another server)
- Text-bared protocol.

Cons:
- does not handle incoming emails, only outgoing.
- vulnerable to ~~scams~~ spam abuse if not properly secured.

pros:
- reliable email delivery across servers.
- multiple recipent's in a single mail.
- Standard protocol compatible with most email services.

Port Secure 465 — SMTP over (SSL)
587 — SMTP (TLS)

Non Secure : 25 — SMTP

(2) POP3 Purpose: Downloading emails

- Download emails from server → local device
- by default, downloaded emails are removed from the server.
- emails are stored locally, so they can be read offline.
- limited sync. b/w devices.
- Can optionally be configured to keep a copy on the server.

working — mail server —POP3→ Computer → Local emails

default non-secure : 110 — POP3
secure : 995 — POP3 over SSL/TLS

Cons:
- limited to a single-device setup, with no sync.
- Changes made locally do not reflect on the server on other devices.

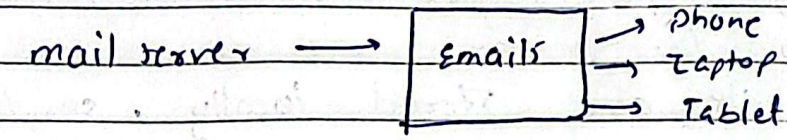
Pros:
- offline access of emails.
- Saves server storage
- Simpler protocol with fewer resources required

(3) IMAP Purpose: Accessing + sync emails.

- email remain on the mail server
- Sync emails across multiple devices
- changes such as:
  read / unread, delete, move to folder
- supports folders & searching.
- requires internet connectivity to access server-stored emails.

Secure 993 - IMAP

Non-S. 143 - IMAP over SSL/TLS



CONS: • requires consistent internet connectivity to access emails.

• consumes more server storage compared to POP3

PROS: • emails remain on server, enable multi-device sync.

• supports advanced folder organization & search ~~facilities~~ functionalities.

• changes made on one device reflect across devices.

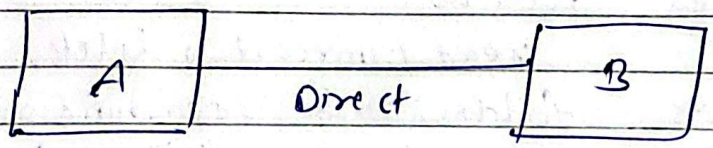
* Peer 2 Peer:-

• No central Server required.

• Peer communication directly with each other

• Each peer can send & receive data

• resources such as files, storage, bandwidth can be shared b/w peers.



diff b/w webRTC & WebSocket-

① webRTC : Web Real-Time-Communication.

It allows real-time, peer-to-peer, communication b/w browsers / devices.

Working 1. signaling server helps two peer discover & establish the connection.
2. connection established then actual media/data can flow directly b/w the peers.

Pros: • Low latency, • Audio & video support,
• P2P communication,
• Data channels (RTC Data channel for sending arbitrary data). ex: File, Chat, Game data, JSON.
• Secure.

Cons: • Complicated, • P2P not always possible bcz NAT/firewall may ~~int~~ interupt connection.
Soln : TURN server (A) — Turn server — (B)
• Not ideal for real-time application

② Web Socket : provides a persistent, two-way communication channel between a client and server.

client ⟷ (persistent connection) ⟷ server

Pros:
- simpler to implement
- Great for real-time events — chat, Notif, Live dashboards, stock updates, online games, collaborative apps.
- Server has centralized control
- Reliable connection: as they normally runs over TCP, providing reliable, ordered delivery.

Cons:
- server becomes a bottleneck
  → as it handles all the traffic from clients
- Not naturally P2P as:
  Client A ⟺ server ⟺ client B
- Not designed specifically for audio/video you can send media data over WebSocket, but WebRTC provides purpose-built APIs and mechanisms for real-time audio/video

Diff b/w Monolithic v/s Microservice (mainly disdv & adv)

* Monolithic (disadv)
  → tight coupled
  → need to scale whole app
  → need consume much memory
  → security issue
  → Overload IDE

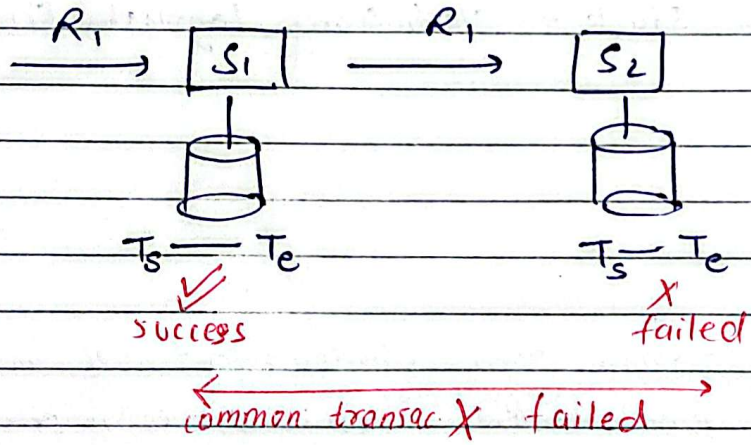
Note: all these are easily handled by microservice Arch.

* ~~Disdv~~ Disadvantages of Microservices:

1) sometimes bcz of loosely coupled we need to call other services or have to change them as well so [LATENCY INC]

2) It's difficult to figure out which other services will break due to changes in one Service.

3) Transaction management:



R → Request
S → Service
Ts → Transaction Start
Te → " end

bcz diff DB it's difficult to manage trans. as one succes & other got failed for same request.

Phases of Microservices (Micro + Services)
→ How can be divided.

① Decomposition:
(a) —decompose→ By Business Capabilities
(b) ⟶ By Sub-domain

② Database :

(a) per service
(b) shared DB

③ Communication :

(a) API , (b) Events , ... many more

④ Integration :

API Gateway, e.g. service to service, event-driven etc.

⑤ So on..... Observability :

Metrics (Prometheus, Grafana),
Logs (ELK Stack - Elastic Search, Logstash, Kibana)
etc.

Decompose :

① Buriness Capabilities :

→ what a buriness does, independent of how the software is implemented. ex: E-commerce.

E-comm System

- Product management
- Order management
- Payment
- Inventory management
- Shipping
- Customer management

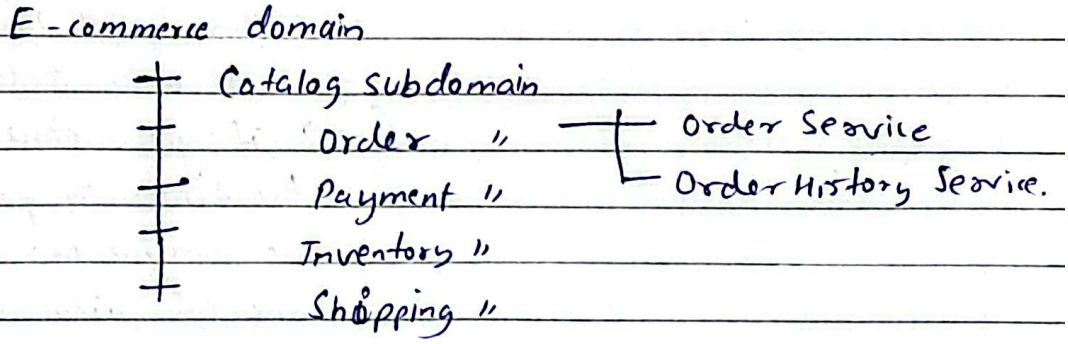
Each capability can become an independent service.

Note : One buss. capa. → one or more microservice

② Sub-Domain

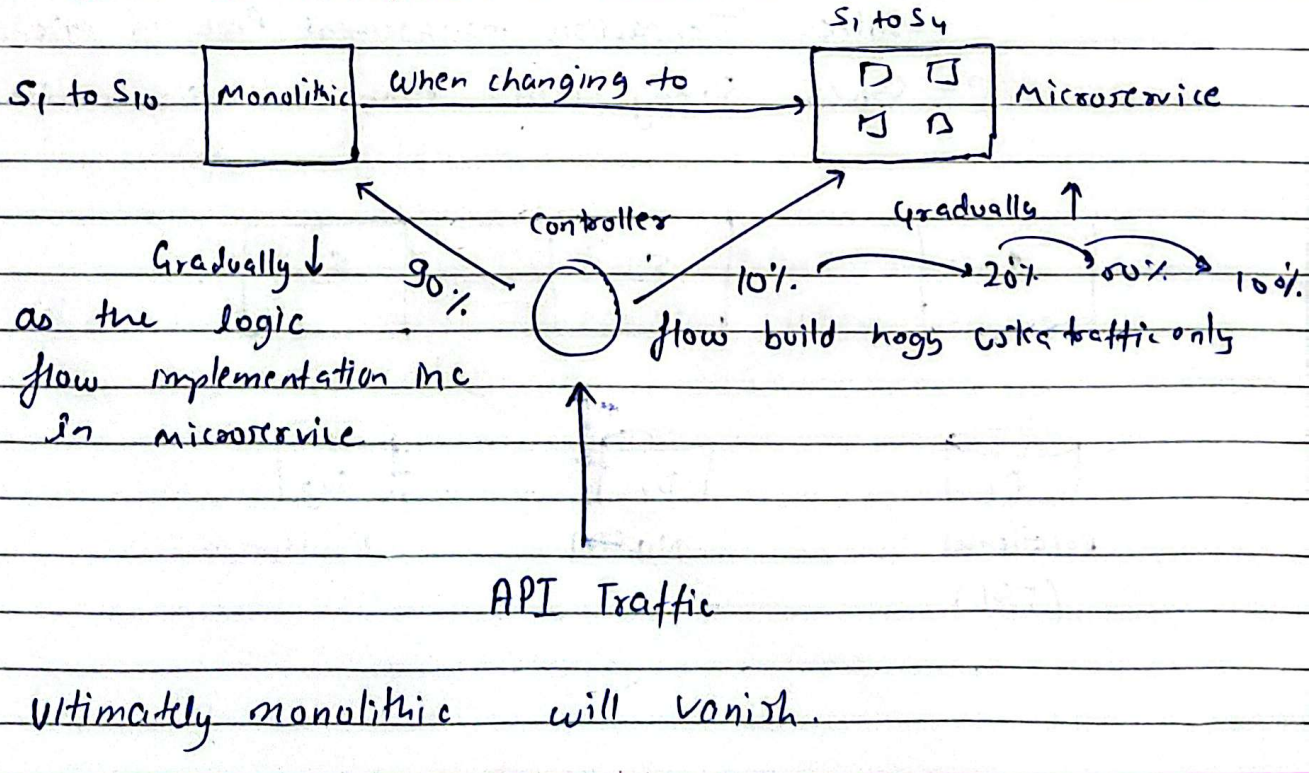
→ this approach comes from Domain-Driven Design (DDD)

→ first divide the overall business domain into subdomains.



Microservices Design Pattern

① STRANGLER Pattern

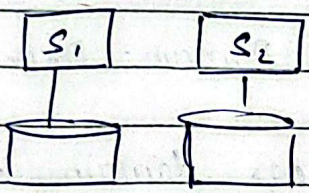


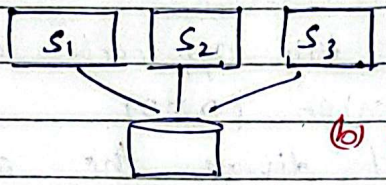
Ultimately monolithic will vanish.

Data management in Microservice:

(a) individual

succesfull for very small services





(b) shared

succesfull

PROS here ← CONS: ① → bcz one service provide too much load then we need to scale whole DB

not successfull that much

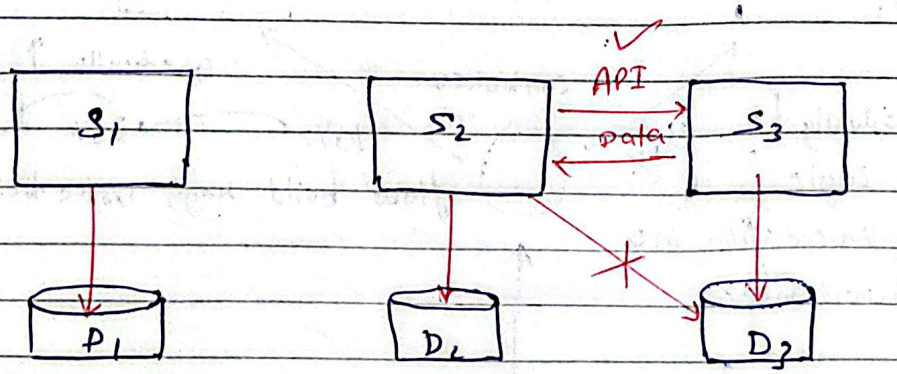
② → if you want to modify the table as per S_1 → not be possible bcz that is also shared with other services.

CONS Here ← PROS: ① Easy to apply JOINS ✓

② Transactional Prop manage ✓

SAGA → Solves Transaction management Prob in Micros.

CQRS → Solves Query Join Prob in Microservice.

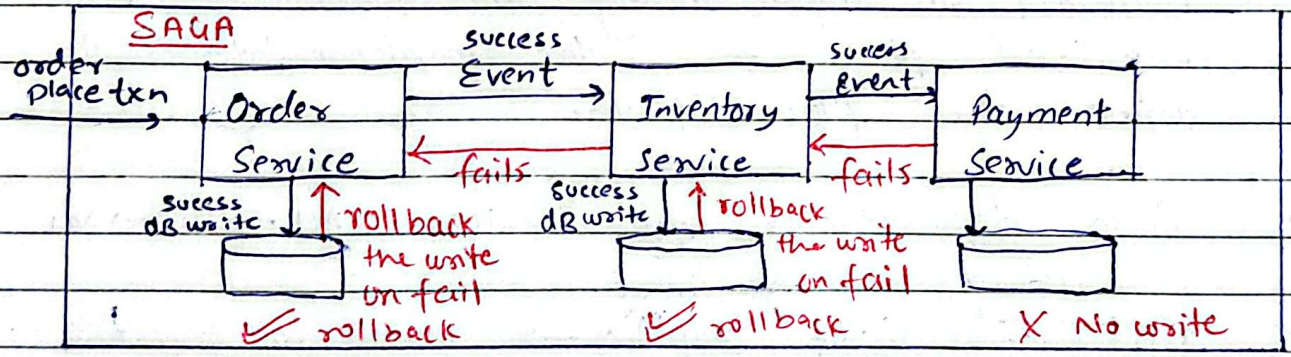


SAGA Pattern: — uses ACID Trans. Framework

→ This pattern is used to maintain data consistency across services.

→ By using a sequence of local transactions that are coordinated using asynchronous messaging.

* → If a local transaction fails, then the Saga executes a series of compensating transactions that undo the changes, which were completed by the preceding local transactions.



* Key challenges:

inventory 2 service change kare simultaneously

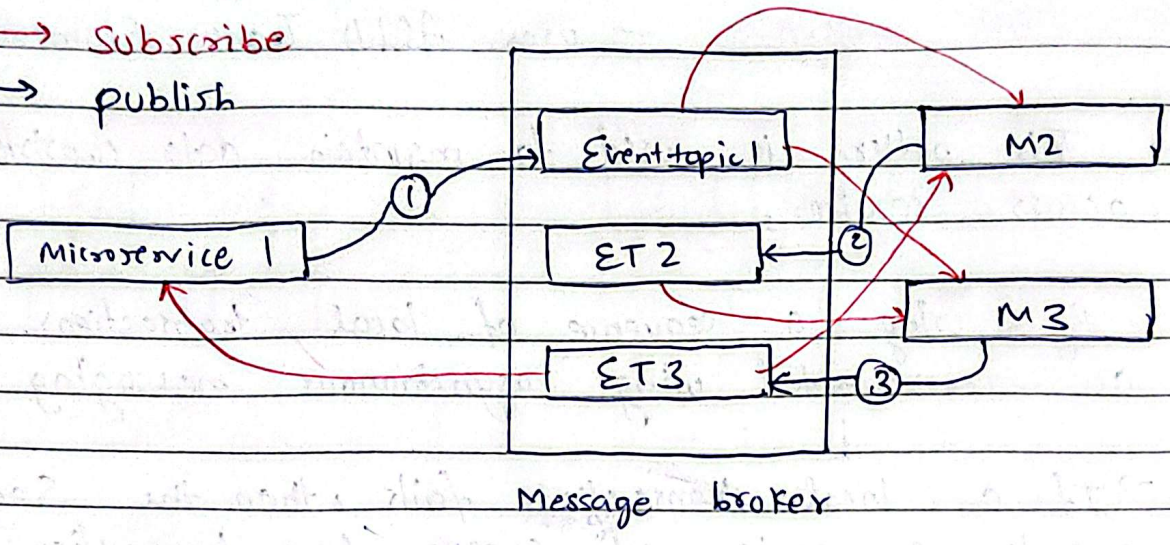
① Lack of isolation b/w sagas.
② Rolling back changes when an error happens within a Saga → long problem of compensation. (real world prob)

* Two ways to implement SAGA:

① Choreography based sagas.
② Orchestration " "

① Choreography Based.

→ Subscribe
→ publish

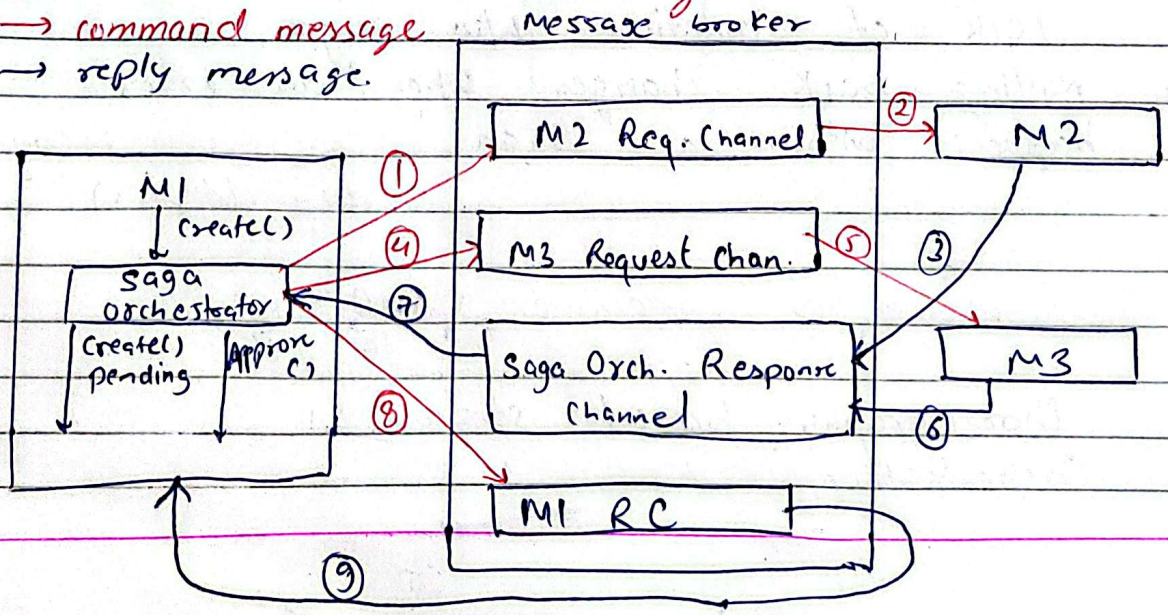


* drawbacks of this implementation

- Difficulty in understanding the flow : As no centralized place for the implementation to define the flow of the saga ..
- Cycle dependencies b/w services : M1 → M2 → M1
- Risk of tight coupling → having to subscribe to all events

② Orchestration based sagas :

→ command message
→ reply message.



→ when microservice 1 receives the create() req, it creates the saga orchestrator.

→ This will set the service req. to the PENDING ~~set~~ state until the saga orchestrator (SO) gets the complete approval.

→ meantime, the SO sends req. commands to both M2, M3 & gets the res. back via the JO response channel in the broker.

→ Bared on the outcome of both responses, the orchestrator will approve or reject the req.

* Key benefits :

• Simpler dependencies → No cyclic dependencies bcz always orchestrator invokes saga participants not vice-versa.

• less coupling → no need to know about events published or the buriness logic implemented by other saga participants. hence improving coupling & business logic.

* Drawbacks :-

① less buriness logic at the saga orchestrator level → allowing to have more decoupled architecture

(2) less Isolation - unlike ACID trans., saga participants commit their local trans. independently before the entire saga completes. This can expose intermediate states, leading to temporary inconsistencies & db anomalies.

Anomalies: (1) lost update — overwritten.
(2) Dirty reads — read in the middle of update
(3) fuzz Reads — two saga read one updated and one in the middle of update

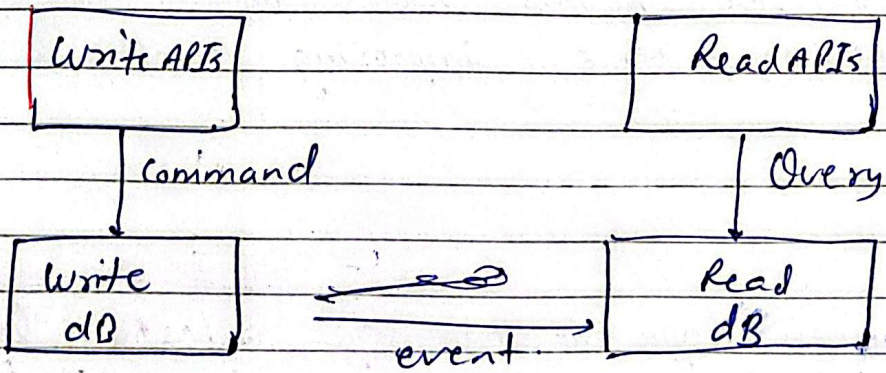
CQRS [Command, Query, Request, Segregation]
from CRUD → Create, U, D ↵ selected

- logic → separate read and write operation into distinct models.

* Two responsibilities :-

(1) commands → which handle state-changing operation such as insert, update, delete.

(2) Queries → which retrieve data without modifying the system state.

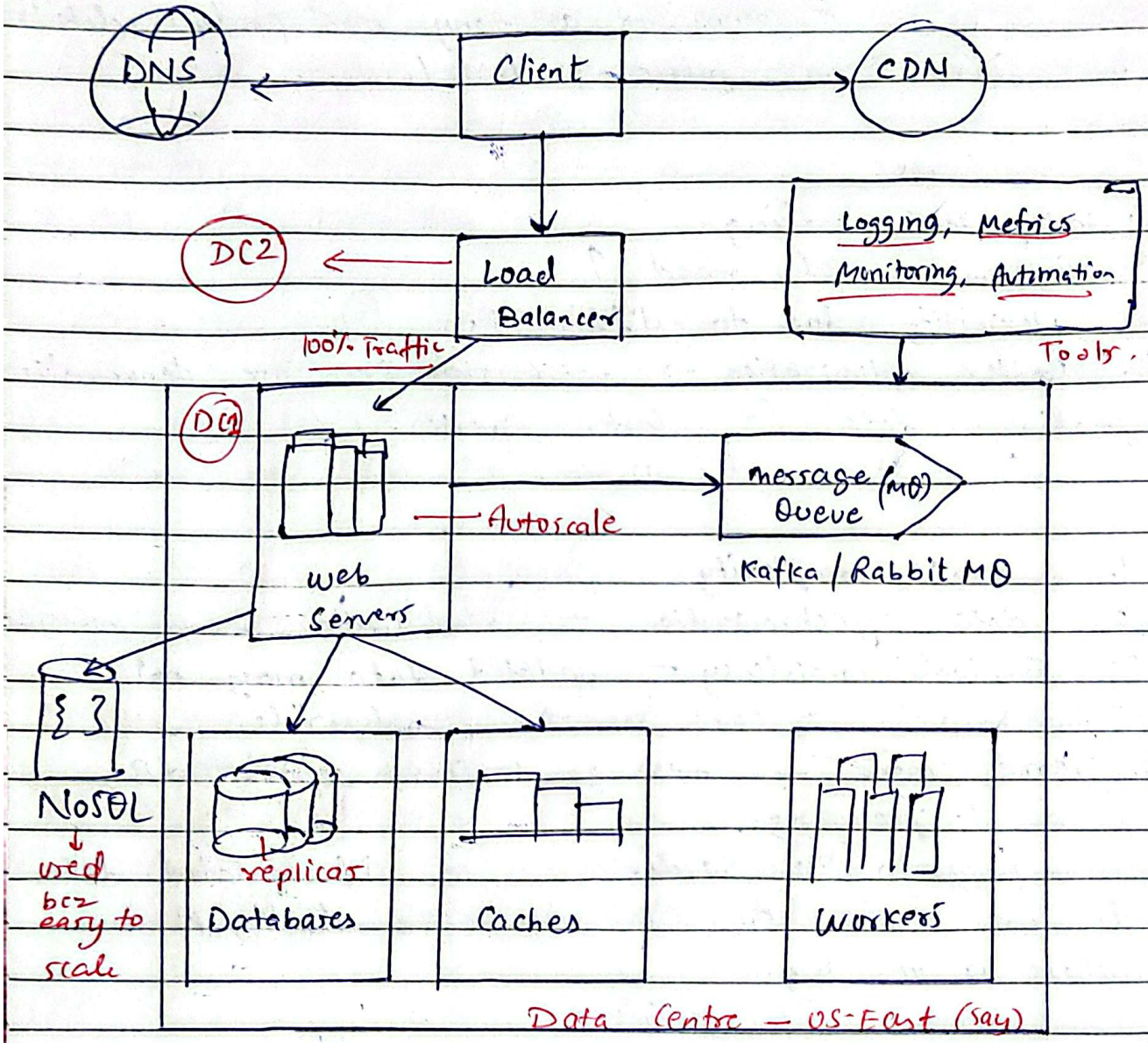


use case – E-comm → as many read product details but fewer user update products/orders.

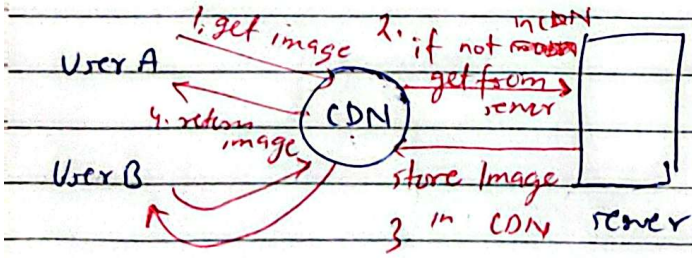
* Advantages :
1. independent scaling
2. Performance of read ↑
3. Flexibility due to diff DBs.
4. Better optimization → read side can use denormalization data for faster queries.

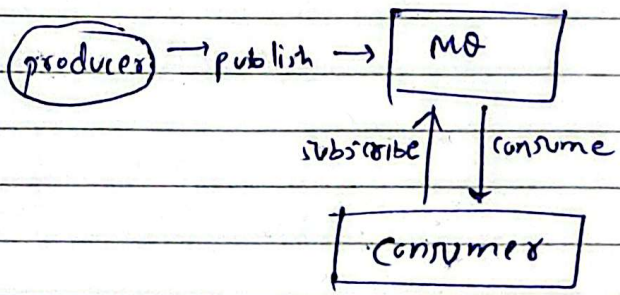
* Drawbacks :
1. inc. complexity
2. data synchronization — read/write becom inconsistent
3. Eventual consistency — updated data may not immediately appear on the read side.
4. More infra — message brokers, separate dB, event processing.
5. debugging is harder — as if updated data is not shown on UI then we don't know where is the bug.

Scale to 1M or 1B same structure just more explanation about dockers, kubernetes, etc...



Autoscaling → means adding or removing web servers automatically based on the traffic load





Note: In the event of any significant outage, we direct all traffic to healthy data centers.

CamScanner

Scale From Zero To Millions Of Users

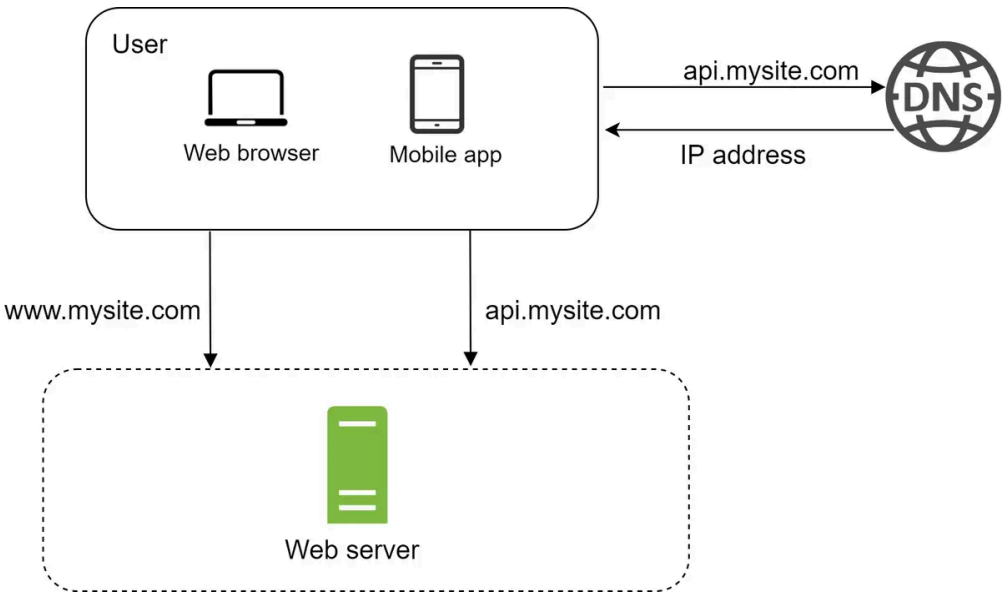
bytebytego.com/courses/system-design-interview/scale-from-zero-to-millions-of-users

02

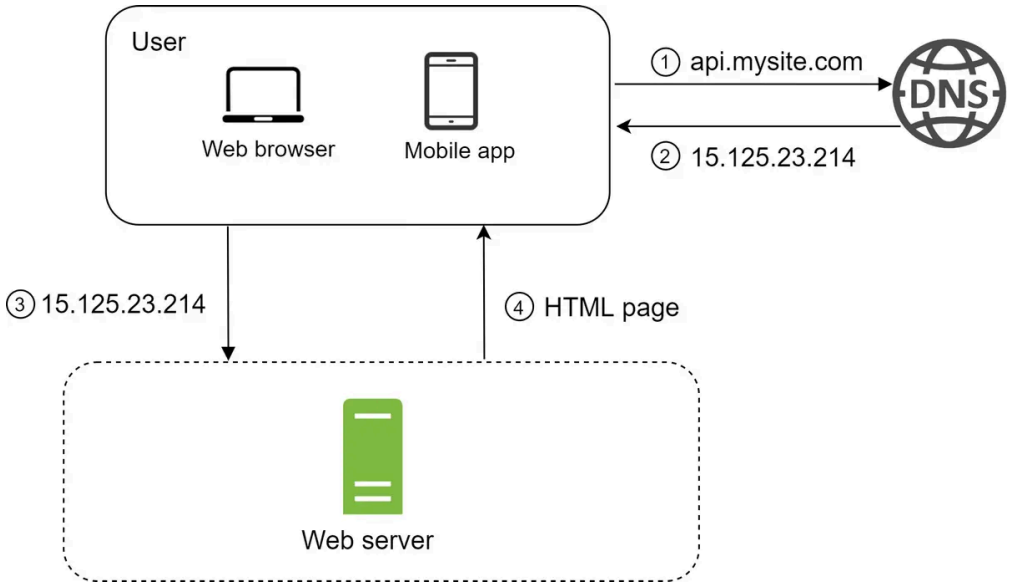
Designing a system that supports millions of users is challenging, and it is a journey that requires continuous refinement and endless improvement. In this chapter, we build a system that supports a single user and gradually scale it up to serve millions of users. After reading this chapter, you will master a handful of techniques that will help you to crack the system design interview questions.

Single server setup

A journey of a thousand miles begins with a single step, and building a complex system is no different. To start with something simple, everything is running on a single server. Figure 1 shows the illustration of a single server setup where everything is running on one server: web app, database, cache, etc.



To understand this setup, it is helpful to investigate the request flow and traffic source. Let us first look at the request flow (Figure 2).



1. Users access websites through domain names, such as api.mysite.com. Usually, the Domain Name System (DNS) is a paid service provided by 3rd parties and not hosted by our servers.

2. Internet Protocol (IP) address is returned to the browser or mobile app. In the example, IP address 15.125.23.214 is returned.

3. Once the IP address is obtained, Hypertext Transfer Protocol (HTTP) [1] requests are sent directly to your web server.

4. The web server returns HTML pages or JSON response for rendering.

Next, let us examine the traffic source. The traffic to your web server comes from two sources: web application and mobile application.

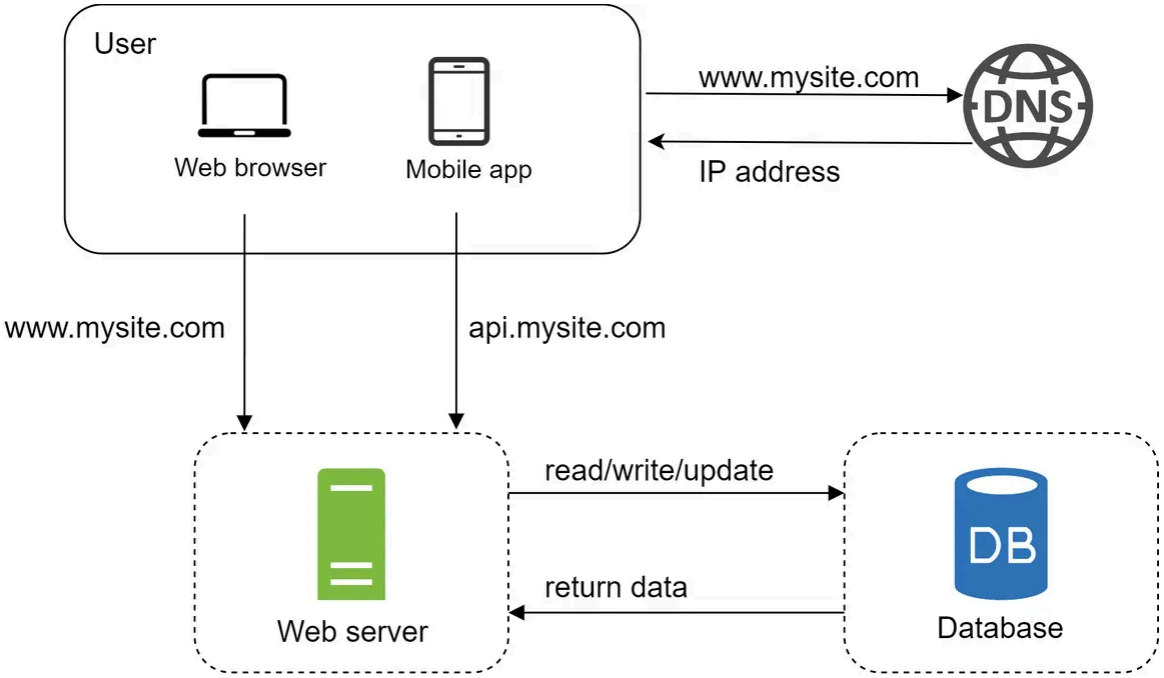
- Web application: it uses a combination of server-side languages (Java, Python, etc.) to handle business logic, storage, etc., and client-side languages (HTML and JavaScript) for presentation.
- Mobile application: HTTP protocol is the communication protocol between the mobile app and the web server. JavaScript Object Notation (JSON) is commonly used API response format to transfer data due to its simplicity. An example of the API response in JSON format is shown below:

GET /users/12 – Retrieve user object for id = 12

```
{
   "id":12,
   "firstName":"John",
   "lastName":"Smith",
   "address":{
      "streetAddress":"21 2nd Street",
      "city":"New York",
      "state":"NY",
      "postalCode":10021
   },
   "phoneNumbers":[
      "212 555-1234",
      "646 555-4567"
   ]
}
```

Database

With the growth of the user base, one server is not enough, and we need multiple servers: one for web/mobile traffic, the other for the database (Figure 3). Separating web/mobile traffic (web tier) and database (data tier) servers allows them to be scaled independently.



Which databases to use?

You can choose between a traditional relational database and a non-relational database. Let us examine their differences.

Relational databases are also called a relational database management system (RDBMS) or SQL database. The most popular ones are MySQL, Oracle database, PostgreSQL, etc. Relational databases represent and store data in tables and rows. You can perform join operations using SQL across different database tables.

Non-Relational databases are also called NoSQL databases. Popular ones are CouchDB, Neo4j, Cassandra, HBase, Amazon DynamoDB, etc. [2]. These databases are grouped into four categories: key-value stores, graph stores, column stores, and document stores. Join operations are generally not supported in non-relational databases.

For most developers, relational databases are the best option because they have been around for over 40 years and historically, they have worked well. However, if relational databases are not suitable for your specific use cases, it is critical to explore beyond relational databases. Non-relational databases might be the right choice if:

- Your application requires super-low latency.
- Your data are unstructured, or you do not have any relational data.
- You only need to serialize and deserialize data (JSON, XML, YAML, etc.).
- You need to store a massive amount of data.

Vertical scaling vs horizontal scaling

Vertical scaling, referred to as “scale up”, means the process of adding more power (CPU, RAM, etc.) to your servers. Horizontal scaling, referred to as “scale-out”, allows you to scale by adding more servers into your pool of resources.

When traffic is low, vertical scaling is a great option, and the simplicity of vertical scaling is its main advantage. Unfortunately, it comes with serious limitations.

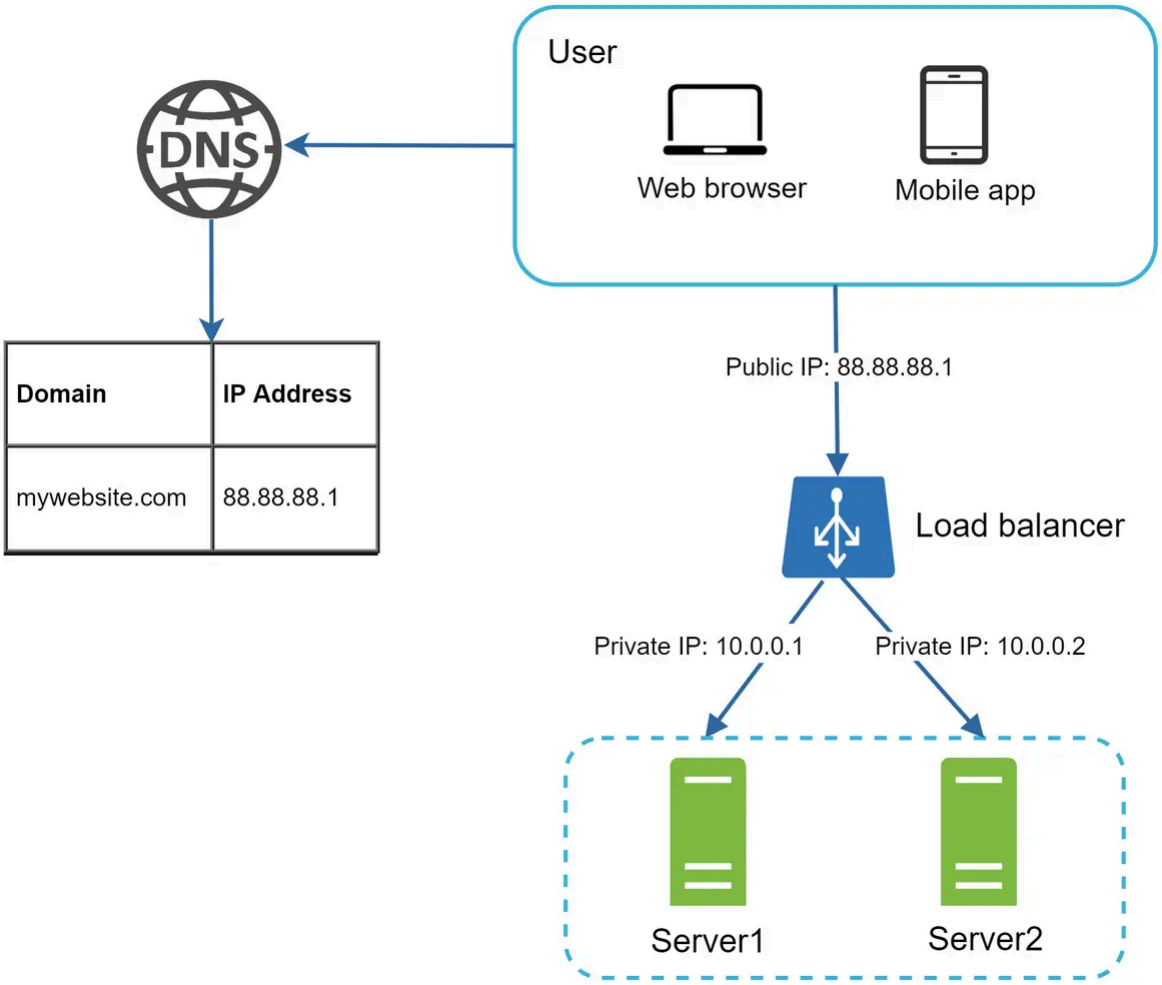
- Vertical scaling has a hard limit. It is impossible to add unlimited CPU and memory to a single server.
- Vertical scaling does not have failover and redundancy. If one server goes down, the website/app goes down with it completely.

Horizontal scaling is more desirable for large scale applications due to the limitations of vertical scaling.

In the previous design, users are connected to the web server directly. Users will unable to access the website if the web server is offline. In another scenario, if many users access the web server simultaneously and it reaches the web server’s load limit, users generally experience slower response or fail to connect to the server. A load balancer is the best technique to address these problems.

Load balancer

A load balancer evenly distributes incoming traffic among web servers that are defined in a load-balanced set. Figure 4 shows how a load balancer works.



As shown in Figure 4, users connect to the public IP of the load balancer directly. With this setup, web servers are unreachable directly by clients anymore. For better security, private IPs are used for communication between servers. A private IP is an IP address reachable only between servers in the same network; however, it is unreachable over the internet. The load balancer communicates with web servers through private IPs.

In Figure 4, after a load balancer and a second web server are added, we successfully solved no failover issue and improved the availability of the web tier. Details are explained below:

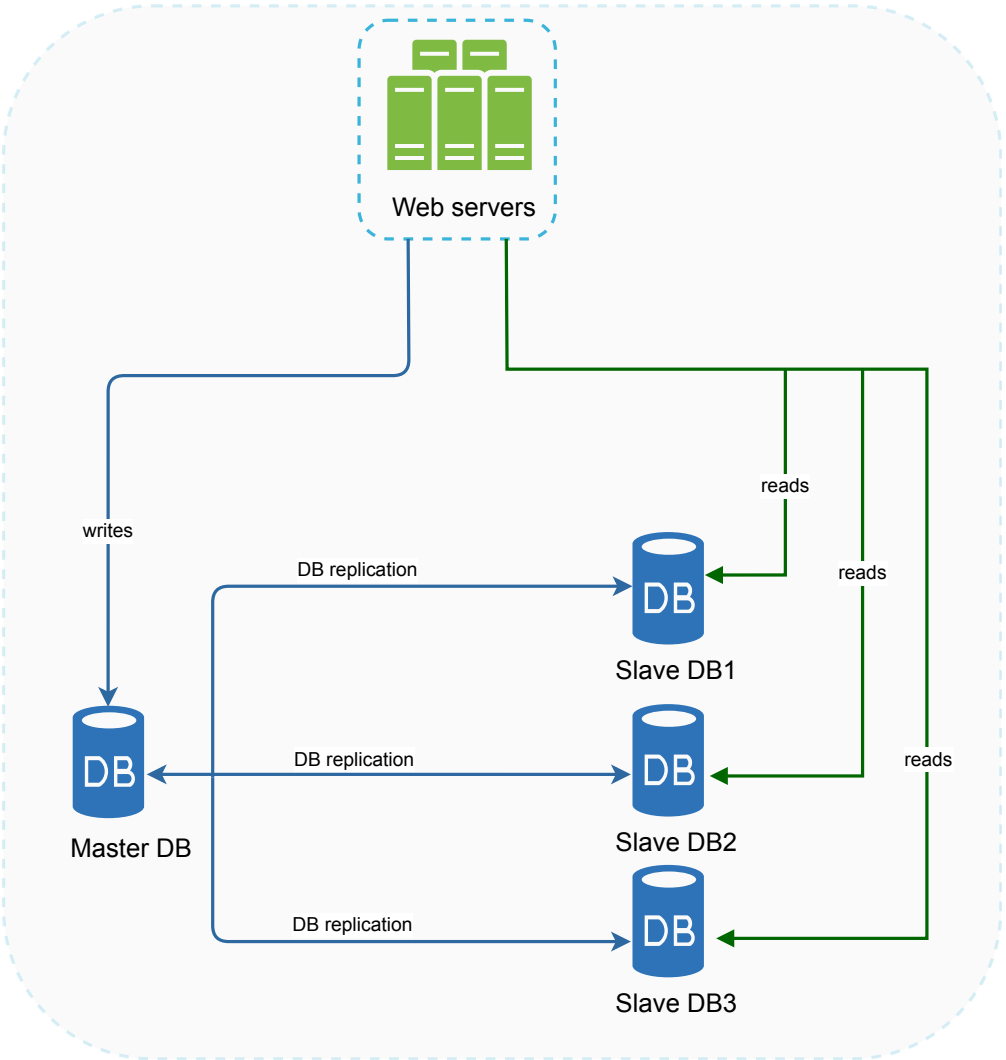
- If server 1 goes offline, all the traffic will be routed to server 2. This prevents the website from going offline. We will also add a new healthy web server to the server pool to balance the load.
- If the website traffic grows rapidly, and two servers are not enough to handle the traffic, the load balancer can handle this problem gracefully. You only need to add more servers to the web server pool, and the load balancer automatically starts to send requests to them.

Now the web tier looks good, what about the data tier? The current design has one database, so it does not support failover and redundancy. Database replication is a common technique to address those problems. Let us take a look.

Database replication

Quoted from Wikipedia: “Database replication can be used in many database management systems, usually with a master/slave relationship between the original (master) and the copies (slaves)” [3].

A master database generally only supports write operations. A slave database gets copies of the data from the master database and only supports read operations. All the data-modifying commands like insert, delete, or update must be sent to the master database. Most applications require a much higher ratio of reads to writes; thus, the number of slave databases in a system is usually larger than the number of master databases. Figure 5 shows a master database with multiple slave databases.



Advantages of database replication:

- Better performance: In the master-slave model, all writes and updates happen in master nodes; whereas, read operations are distributed across slave nodes. This model improves performance because it allows more queries to be processed in parallel.

- Reliability: If one of your database servers is destroyed by a natural disaster, such as a typhoon or an earthquake, data is still preserved. You do not need to worry about data loss because data is replicated across multiple locations.

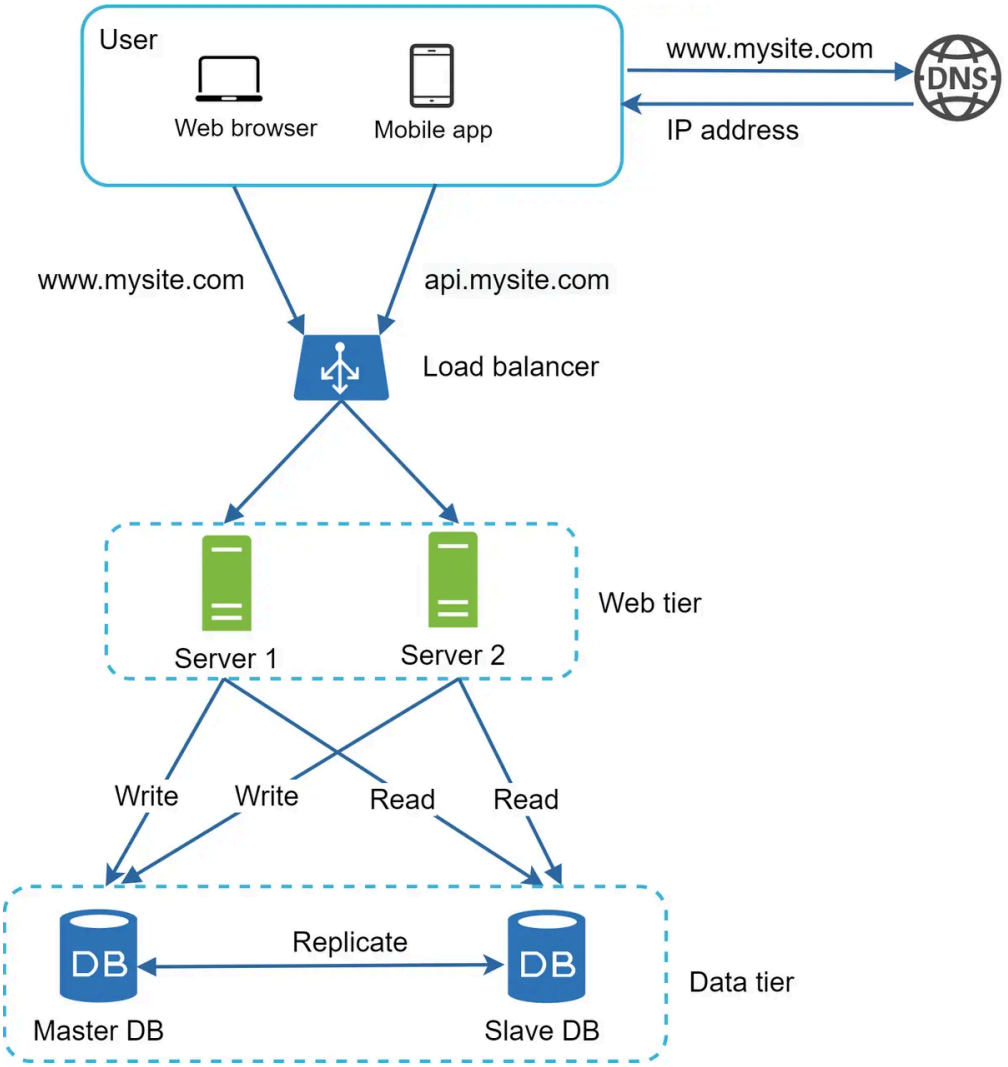
- High availability: By replicating data across different locations, your website remains in operation even if a database is offline as you can access data stored in another database server.

In the previous section, we discussed how a load balancer helped to improve system availability. We ask the same question here: what if one of the databases goes offline? The architectural design discussed in Figure 5 can handle this case:

- If only one slave database is available and it goes offline, read operations will be directed to the master database temporarily. As soon as the issue is found, a new slave database will replace the old one. In case multiple slave databases are available, read operations are redirected to other healthy slave databases. A new database server will replace the old one.

- If the master database goes offline, a slave database will be promoted to be the new master. All the database operations will be temporarily executed on the new master database. A new slave database will replace the old one for data replication immediately. In production systems, promoting a new master is more complicated as the data in a slave database might not be up to date. The missing data needs to be updated by running data recovery scripts. Although some other replication methods like multi-masters and circular replication could help, those setups are more complicated; and their discussions are beyond the scope of this course. Interested readers should refer to the listed reference materials [4] [5].

Figure 6 shows the system design after adding the load balancer and database replication.



Let us take a look at the design:

- A user gets the IP address of the load balancer from DNS.
- A user connects the load balancer with this IP address.
- The HTTP request is routed to either Server 1 or Server 2.
- A web server reads user data from a slave database.
- A web server routes any data-modifying operations to the master database. This includes write, update, and delete operations.

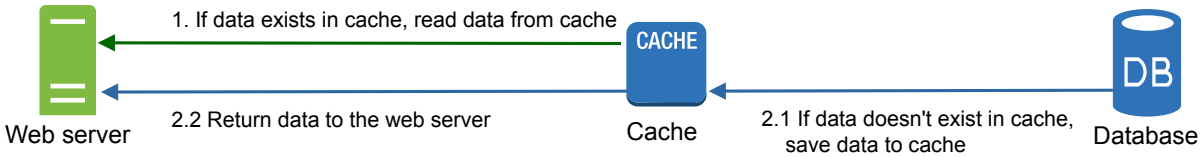
Now, you have a solid understanding of the web and data tiers, it is time to improve the load/response time. This can be done by adding a cache layer and shifting static content (JavaScript/CSS/image/video files) to the content delivery network (CDN).

Cache

A cache is a temporary storage area that stores the result of expensive responses or frequently accessed data in memory so that subsequent requests are served more quickly. As illustrated in Figure 6, every time a new web page loads, one or more database calls are executed to fetch data. The application performance is greatly affected by calling the database repeatedly. The cache can mitigate this problem.

Cache tier

The cache tier is a temporary data store layer, much faster than the database. The benefits of having a separate cache tier include better system performance, ability to reduce database workloads, and the ability to scale the cache tier independently. Figure 7 shows a possible setup of a cache server:



After receiving a request, a web server first checks if the cache has the available response. If it has, it sends data back to the client. If not, it queries the database, stores the response in cache, and sends it back to the client. This caching strategy is called a read-through cache. Other caching strategies are available depending on the data type, size, and access patterns. A previous study explains how different caching strategies work [6].

Interacting with cache servers is simple because most cache servers provide APIs for common programming languages. The following code snippet shows typical Memcached APIs:

```
SECONDS = 1
cache.set('myKey, 'hi there', 3600 * SECONDS)
cache.get('myKey')
```

Considerations for using cache

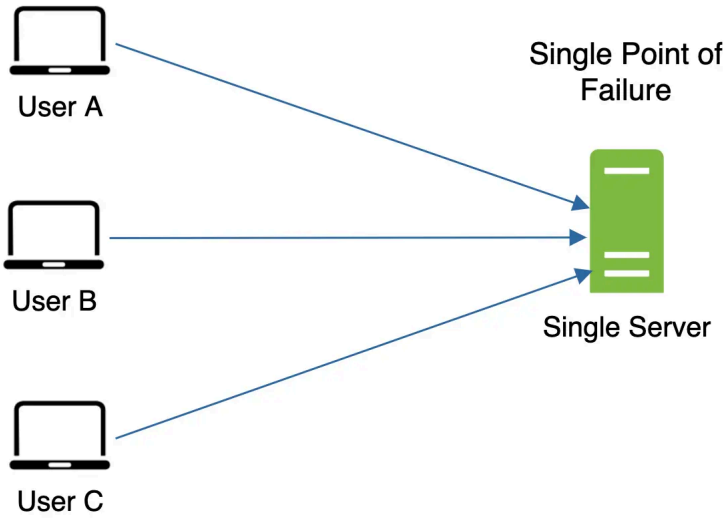
Here are a few considerations for using a cache system:

- Decide when to use cache. Consider using cache when data is read frequently but modified infrequently. Since cached data is stored in volatile memory, a cache server is not ideal for persisting data. For instance, if a cache server restarts, all the data in memory is lost. Thus, important data should be saved in persistent data stores.

- Expiration policy. It is a good practice to implement an expiration policy. Once cached data is expired, it is removed from the cache. When there is no expiration policy, cached data will be stored in the memory permanently. It is advisable not to make the expiration date too short as this will cause the system to reload data from the database too frequently. Meanwhile, it is advisable not to make the expiration date too long as the data can become stale.

- Consistency: This involves keeping the data store and the cache in sync. Inconsistency can happen because data-modifying operations on the data store and cache are not in a single transaction. When scaling across multiple regions, maintaining consistency between the data store and cache is challenging. For further details, refer to the paper titled “Scaling Memcache at Facebook” published by Facebook [7].

- Mitigating failures: A single cache server represents a potential single point of failure (SPOF), defined in Wikipedia as follows: “A single point of failure (SPOF) is a part of a system that, if it fails, will stop the entire system from working” [8]. As a result, multiple cache servers across different data centers are recommended to avoid SPOF. Another recommended approach is to overprovision the required memory by certain percentages. This provides a buffer as the memory usage increases.



Eviction Policy: Once the cache is full, any requests to add items to the cache might cause existing items to be removed. This is called cache eviction. Least-recently-used (LRU) is the most popular cache eviction policy. Other eviction policies, such as the Least Frequently Used (LFU) or First in First Out (FIFO), can be adopted to satisfy different use cases.

Content delivery network (CDN)

A CDN is a network of geographically dispersed servers used to deliver static content. CDN servers cache static content like images, videos, CSS, JavaScript files, etc.

Dynamic content caching is a relatively new concept and beyond the scope of this course. It enables the caching of HTML pages that are based on request path, query strings, cookies, and request headers. Refer to the article mentioned in reference material [9] for more about this. This course focuses on how to use CDN to cache static content.

Here is how CDN works at the high-level: when a user visits a website, a CDN server closest to the user will deliver static content. Intuitively, the further users are from CDN servers, the slower the website loads. For example, if CDN servers are in San Francisco, users in Los Angeles will get content faster than users in Europe. Figure 9 is a great example that shows how CDN improves load time.

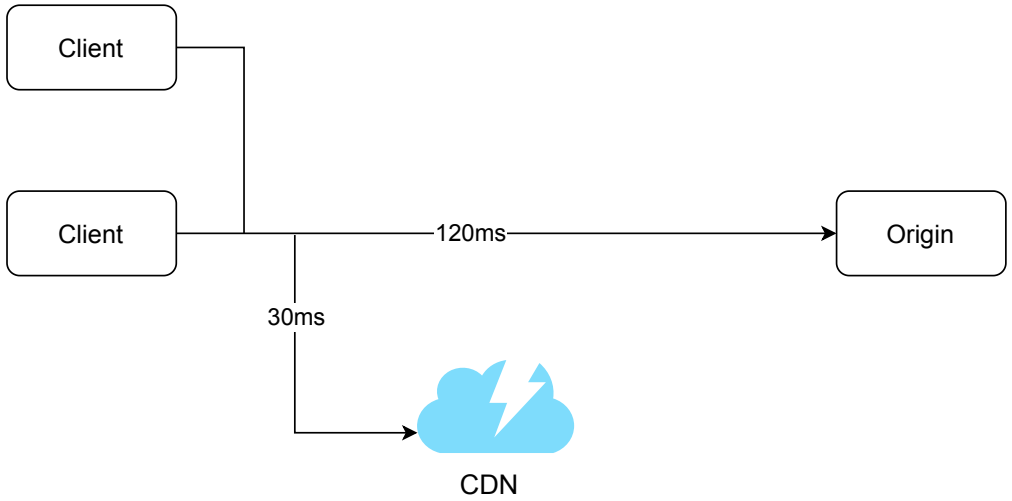
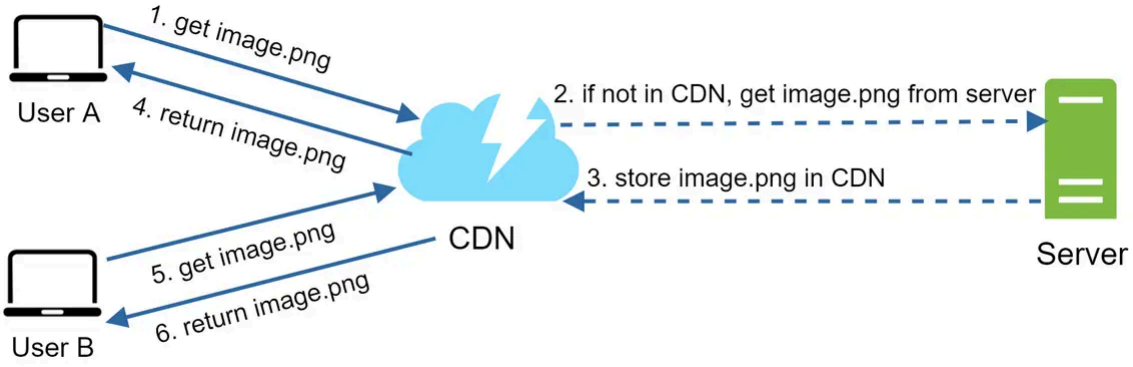


Figure 10 demonstrates the CDN workflow.



1. User A tries to get image.png by using an image URL. The URL's domain is provided by the CDN provider. The following two image URLs are samples used to demonstrate what image URLs look like on Amazon and Akamai CDNs:

- `https://mysite.cloudfront.net/logo.jpg`
- `https://mysite.akamai.com/image-manager/img/logo.jpg`

2. If the CDN server does not have image.png in the cache, the CDN server requests the file from the origin, which can be a web server or online storage like Amazon S3.

3. The origin returns image.png to the CDN server, which includes optional HTTP header Time-to-Live (TTL) which describes how long the image is cached.

4. The CDN caches the image and returns it to User A. The image remains cached in the CDN until the TTL expires.

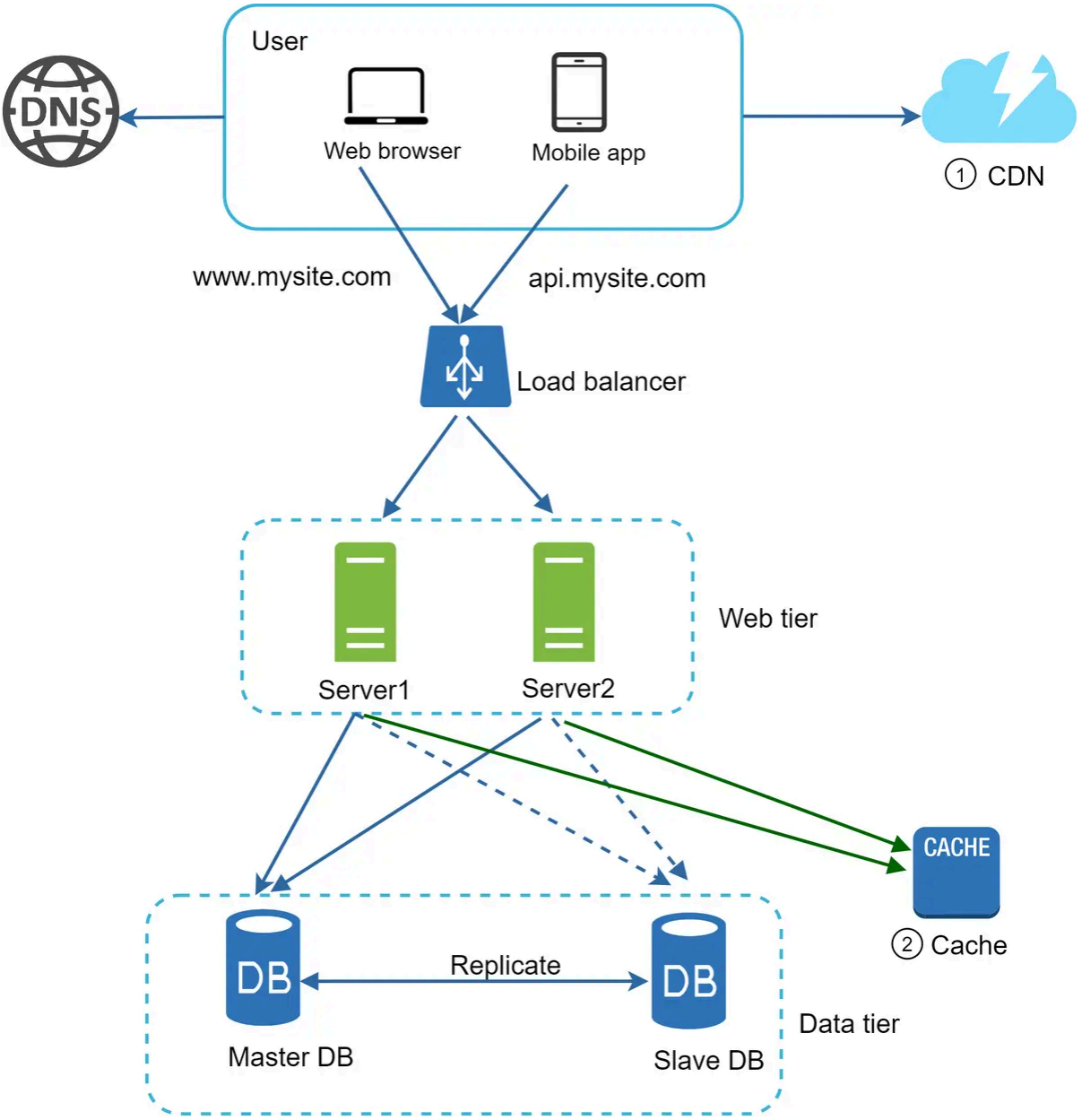
5. User B sends a request to get the same image.

6. The image is returned from the cache as long as the TTL has not expired.

Considerations of using a CDN

- Cost: CDNs are run by third-party providers, and you are charged for data transfers in and out of the CDN. Caching infrequently used assets provides no significant benefits so you should consider moving them out of the CDN.
- Setting an appropriate cache expiry: For time-sensitive content, setting a cache expiry time is important. The cache expiry time should neither be too long nor too short. If it is too long, the content might no longer be fresh. If it is too short, it can cause repeat reloading of content from origin servers to the CDN.
- CDN fallback: You should consider how your website/application copes with CDN failure. If there is a temporary CDN outage, clients should be able to detect the problem and request resources from the origin.
- Invalidating files: You can remove a file from the CDN before it expires by performing one of the following operations:
- Invalidate the CDN object using APIs provided by CDN vendors.
- Use object versioning to serve a different version of the object. To version an object, you can add a parameter to the URL, such as a version number. For example, version number 2 is added to the query string: image.png?v=2.

Figure 11 shows the design after the CDN and cache are added.



1. Static assets (JS, CSS, images, etc.,) are no longer served by web servers. They are fetched from the CDN for better performance.

2. The database load is lightened by caching data.

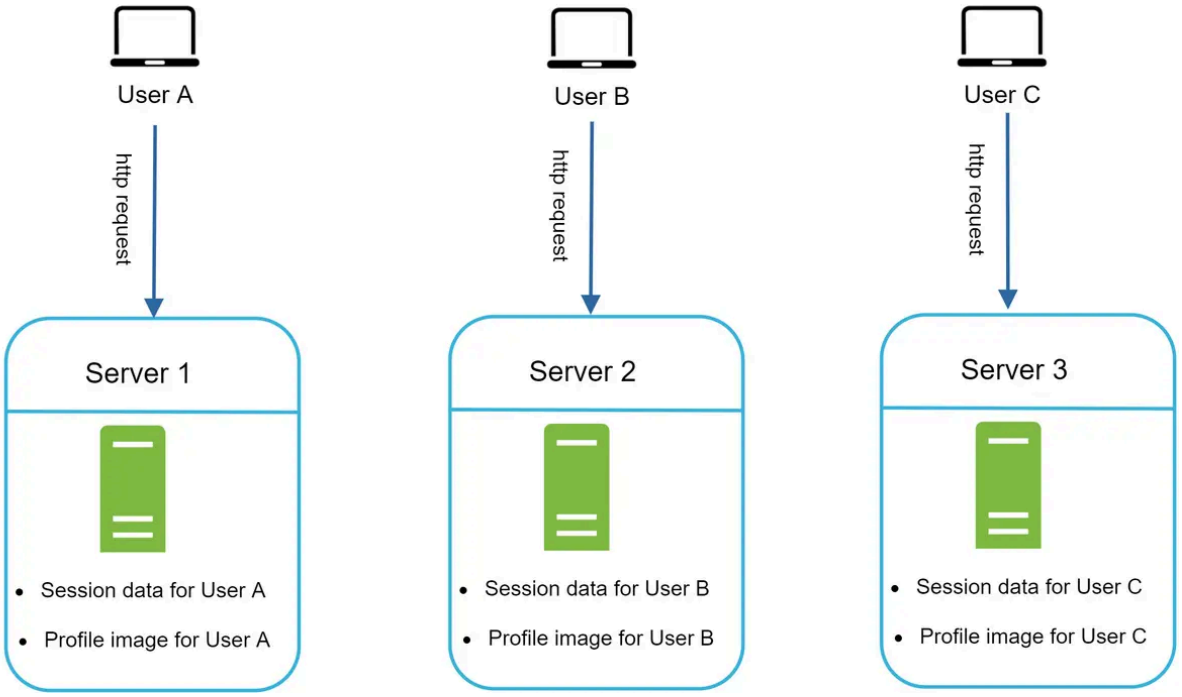
Stateless web tier

Now it is time to consider scaling the web tier horizontally. For this, we need to move state (for instance user session data) out of the web tier. A good practice is to store session data in the persistent storage such as relational database or NoSQL. Each web server in the cluster can access state data from databases. This is called stateless web tier.

Stateful architecture

A stateful server and stateless server has some key differences. A stateful server remembers client data (state) from one request to the next. A stateless server keeps no state information.

Figure 12 shows an example of a stateful architecture.

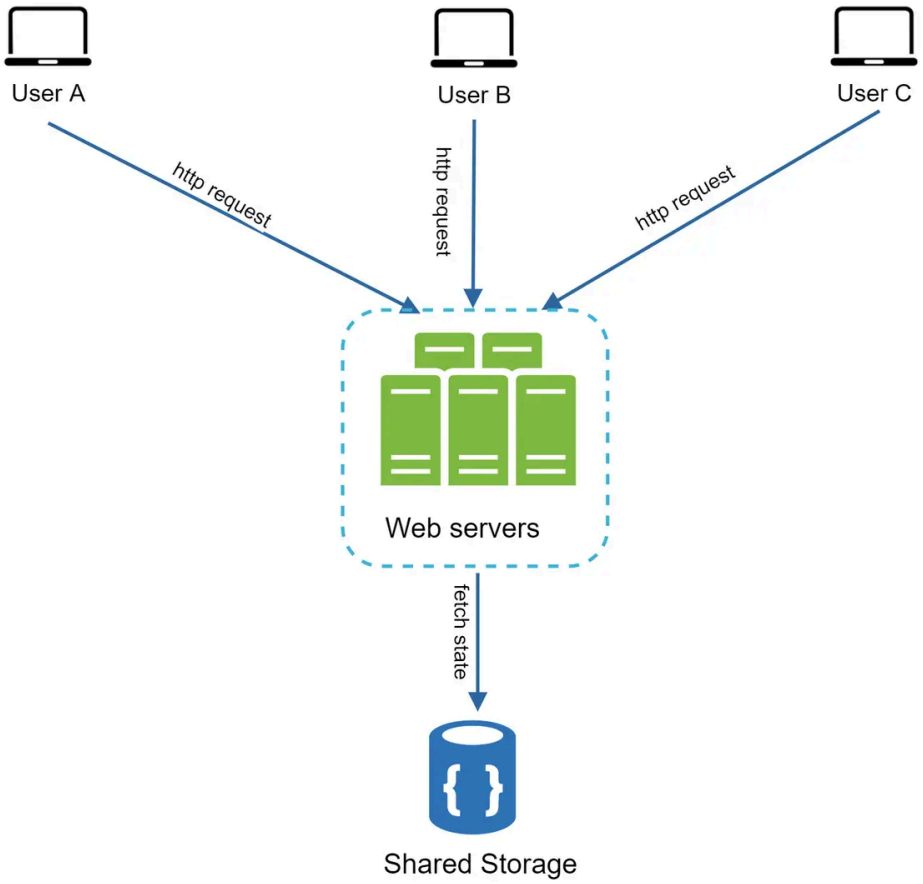


In Figure 12, user A's session data and profile image are stored in Server 1. To authenticate User A, HTTP requests must be routed to Server 1. If a request is sent to other servers like Server 2, authentication would fail because Server 2 does not contain User A's session data. Similarly, all HTTP requests from User B must be routed to Server 2; all requests from User C must be sent to Server 3.

The issue is that every request from the same client must be routed to the same server. This can be done with sticky sessions in most load balancers [10]; however, this adds the overhead. Adding or removing servers is much more difficult with this approach. It is also challenging to handle server failures.

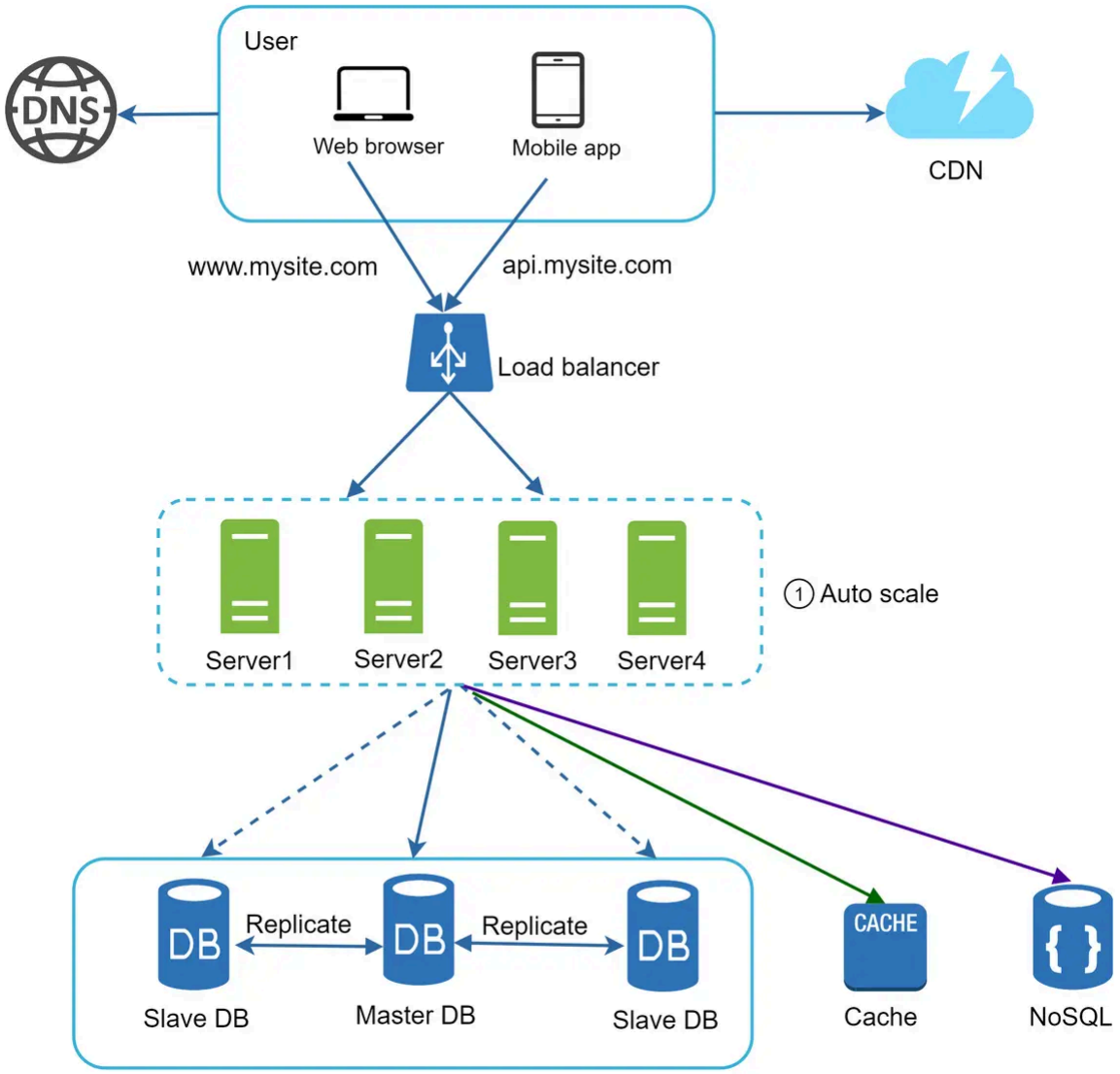
Stateless architecture

Figure 13 shows the stateless architecture.



In this stateless architecture, HTTP requests from users can be sent to any web servers, which fetch state data from a shared data store. State data is stored in a shared data store and kept out of web servers. A stateless system is simpler, more robust, and scalable.

Figure 14 shows the updated design with a stateless web tier.

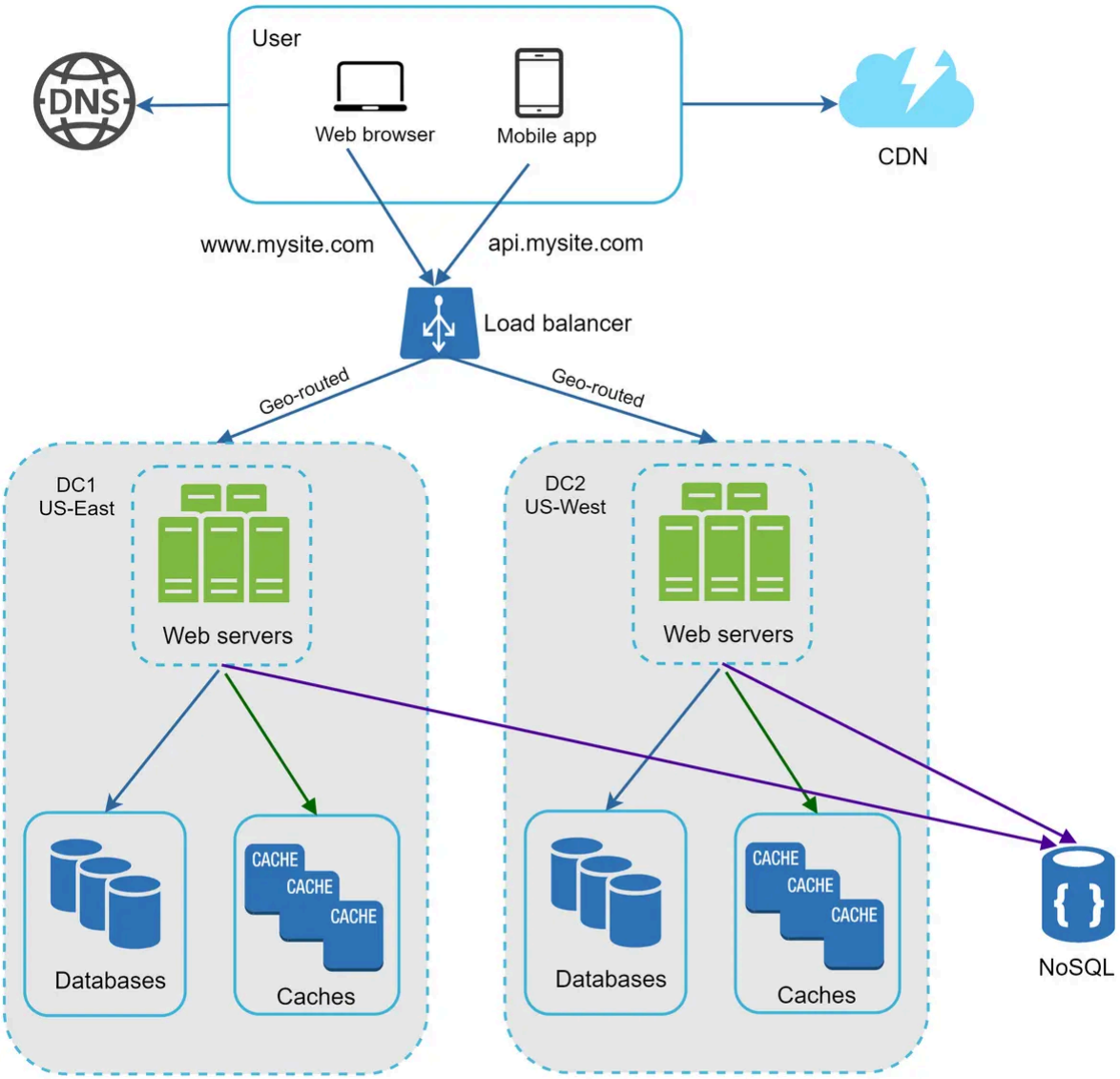


In Figure 14, we move the session data out of the web tier and store them in the persistent data store. The shared data store could be a relational database, Memcached/Redis, NoSQL, etc. The NoSQL data store is chosen as it is easy to scale. Autoscaling means adding or removing web servers automatically based on the traffic load. After the state data is removed out of web servers, auto-scaling of the web tier is easily achieved by adding or removing servers based on traffic load.

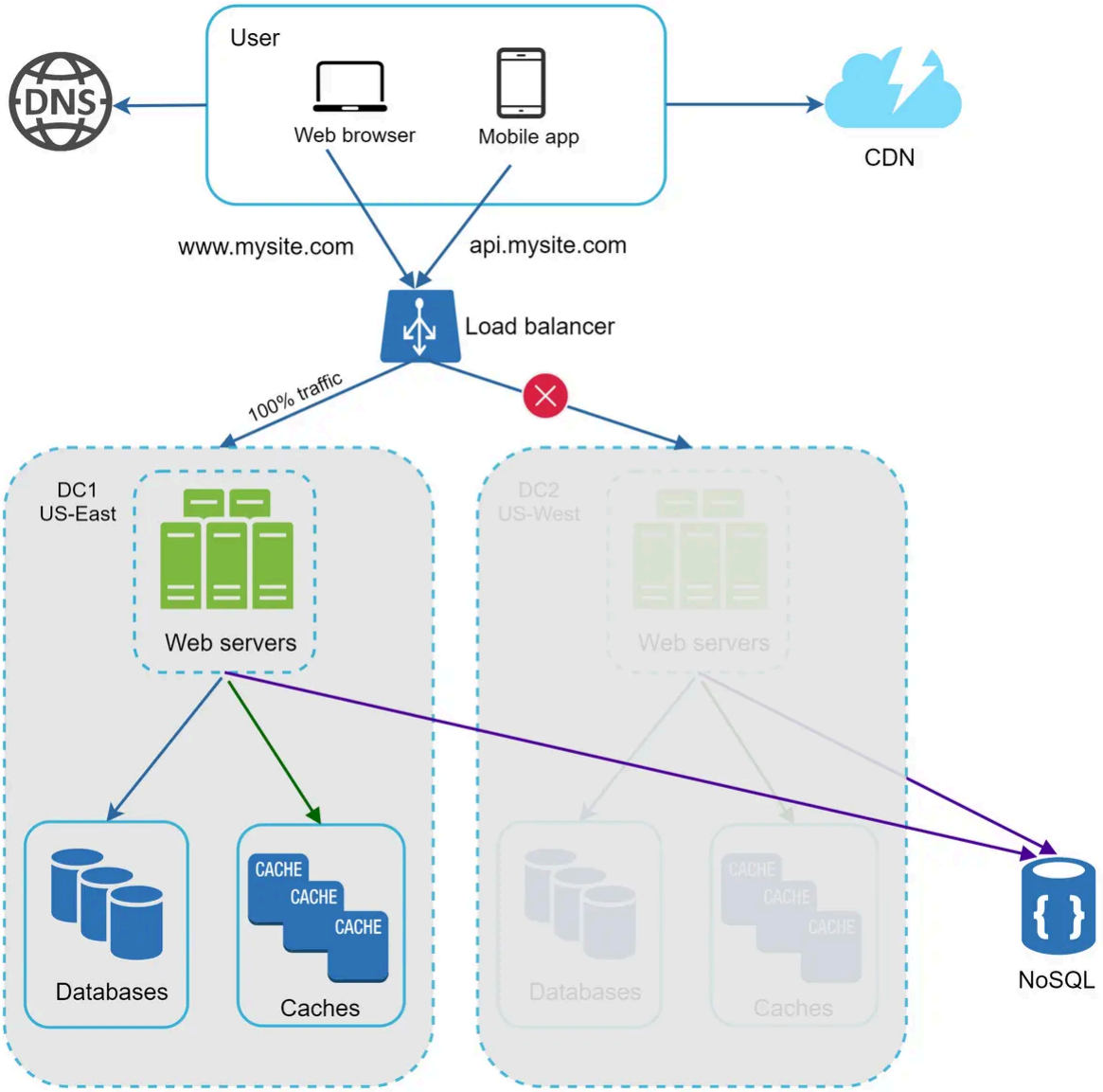
Your website grows rapidly and attracts a significant number of users internationally. To improve availability and provide a better user experience across wider geographical areas, supporting multiple data centers is crucial.

Data centers

Figure 15 shows an example setup with two data centers. In normal operation, users are geoDNS-routed, also known as geo-routed, to the closest data center, with a split traffic of *x%* in US-East and *(100 – x)%* in US-West. geoDNS is a DNS service that allows domain names to be resolved to IP addresses based on the location of a user.



In the event of any significant data center outage, we direct all traffic to a healthy data center. In Figure 16, data center 2 (US-West) is offline, and 100% of the traffic is routed to data center 1 (US-East).



Several technical challenges must be resolved to achieve multi-data center setup:

- Traffic redirection: Effective tools are needed to direct traffic to the correct data center. GeoDNS can be used to direct traffic to the nearest data center depending on where a user is located.
- Data synchronization: Users from different regions could use different local databases or caches. In failover cases, traffic might be routed to a data center where data is unavailable. A common strategy is to replicate data across multiple data centers. A previous study shows how Netflix implements asynchronous multi-data center replication [11].
- Test and deployment: With multi-data center setup, it is important to test your website/application at different locations. Automated deployment tools are vital to keep services consistent through all the data centers [11].

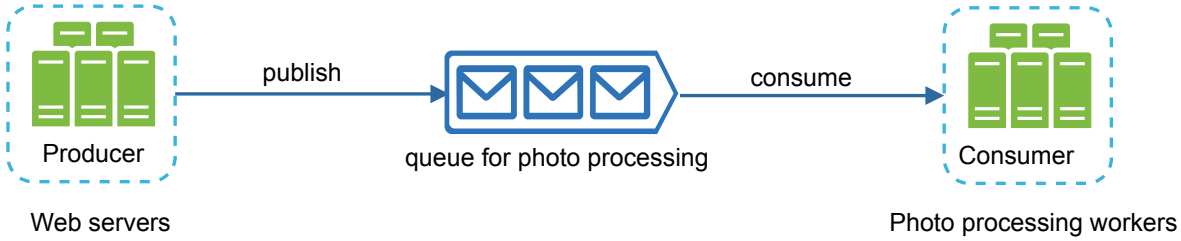
To further scale our system, we need to decouple different components of the system so they can be scaled independently. Messaging queue is a key strategy employed by many real-world distributed systems to solve this problem.

Message queue

A message queue is a durable component, stored in memory, that supports asynchronous communication. It serves as a buffer and distributes asynchronous requests. The basic architecture of a message queue is simple. Input services, called producers/publishers, create messages, and publish them to a message queue. Other services or servers, called consumers/subscribers, connect to the queue, and perform actions defined by the messages. The model is shown in Figure 17.

Decoupling makes the message queue a preferred architecture for building a scalable and reliable application. With the message queue, the producer can post a message to the queue when the consumer is unavailable to process it. The consumer can read messages from the queue even when the producer is unavailable.

Consider the following use case: your application supports photo customization, including cropping, sharpening, blurring, etc. Those customization tasks take time to complete. In Figure 18, web servers publish photo processing jobs to the message queue. Photo processing workers pick up jobs from the message queue and asynchronously perform photo customization tasks. The producer and the consumer can be scaled independently. When the size of the queue becomes large, more workers are added to reduce the processing time. However, if the queue is empty most of the time, the number of workers can be reduced.



Logging, metrics, automation

When working with a small website that runs on a few servers, logging, metrics, and automation support are good practices but not a necessity. However, now that your site has grown to serve a large business, investing in those tools is essential.

Logging: Monitoring error logs is important because it helps to identify errors and problems in the system. You can monitor error logs at per server level or use tools to aggregate them to a centralized service for easy search and viewing.

Metrics: Collecting different types of metrics help us to gain business insights and understand the health status of the system. Some of the following metrics are useful:

- Host level metrics: CPU, Memory, disk I/O, etc.
- Aggregated level metrics: for example, the performance of the entire database tier, cache tier, etc.
- Key business metrics: daily active users, retention, revenue, etc.

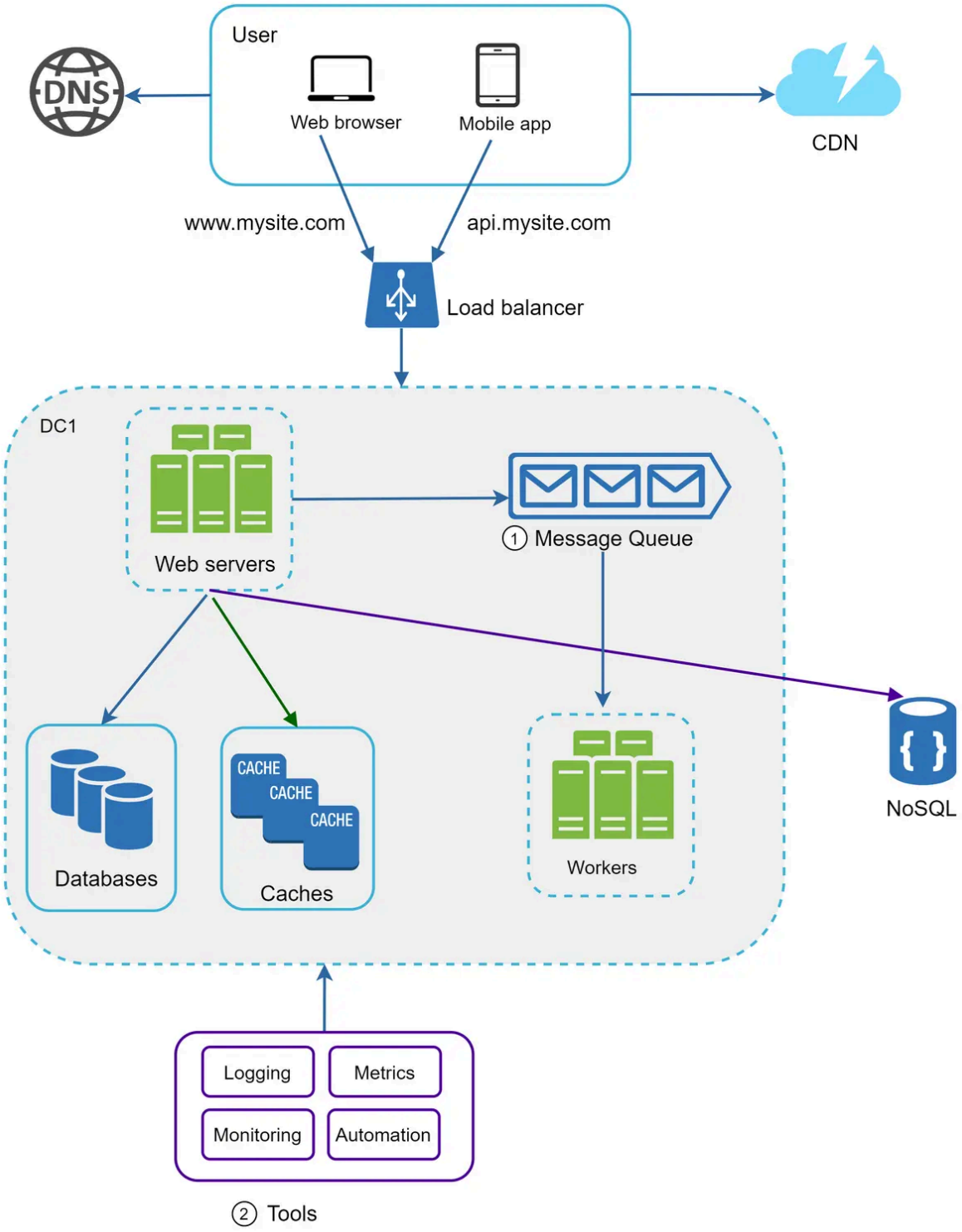
Automation: When a system gets big and complex, we need to build or leverage automation tools to improve productivity. Continuous integration is a good practice, in which each code check-in is verified through automation, allowing teams to detect problems early. Besides, automating your build, test, deploy process, etc. could improve developer productivity significantly.

Adding message queues and different tools

Figure 19 shows the updated design. Due to the space constraint, only one data center is shown in the figure.

1. The design includes a message queue, which helps to make the system more loosely coupled and failure resilient.

2. Logging, monitoring, metrics, and automation tools are included.



As the data grows every day, your database gets more overloaded. It is time to scale the data tier.

Database scaling

There are two broad approaches for database scaling: vertical scaling and horizontal scaling.

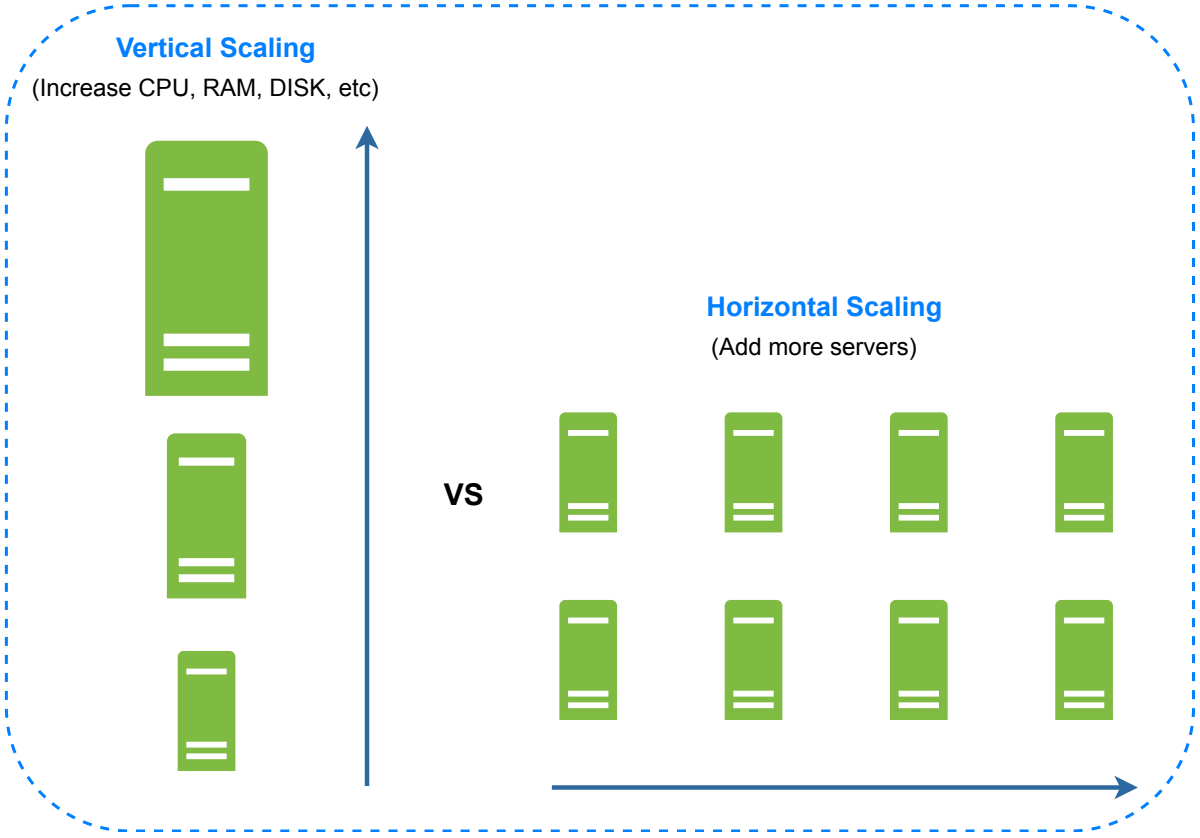
Vertical scaling

Vertical scaling, also known as scaling up, is the scaling by adding more power (CPU, RAM, DISK, etc.) to an existing machine. There are some powerful database servers. According to Amazon Relational Database Service (RDS) [12], you can get a database server with 24 TB of RAM. This kind of powerful database server could store and handle lots of data. For example, stackoverflow.com in 2013 had over 10 million monthly unique visitors, but it only had 1 master database [13]. However, vertical scaling comes with some serious drawbacks:

- You can add more CPU, RAM, etc. to your database server, but there are hardware limits. If you have a large user base, a single server is not enough.
- Greater risk of single point of failures.
- The overall cost of vertical scaling is high. Powerful servers are much more expensive.

Horizontal scaling

Horizontal scaling, also known as sharding, is the practice of adding more servers. Figure 20 compares vertical scaling with horizontal scaling.



Sharding separates large databases into smaller, more easily managed parts called shards. Each shard shares the same schema, though the actual data on each shard

is unique to the shard.

Figure 21 shows an example of sharded databases. User data is allocated to a database server based on user IDs. Anytime you access data, a hash function is used to find the corresponding shard. In our example, *user_id % 4* is used as the hash function. If the result equals to 0, shard 0 is used to store and fetch data. If the result equals to 1, shard 1 is used. The same logic applies to other shards.

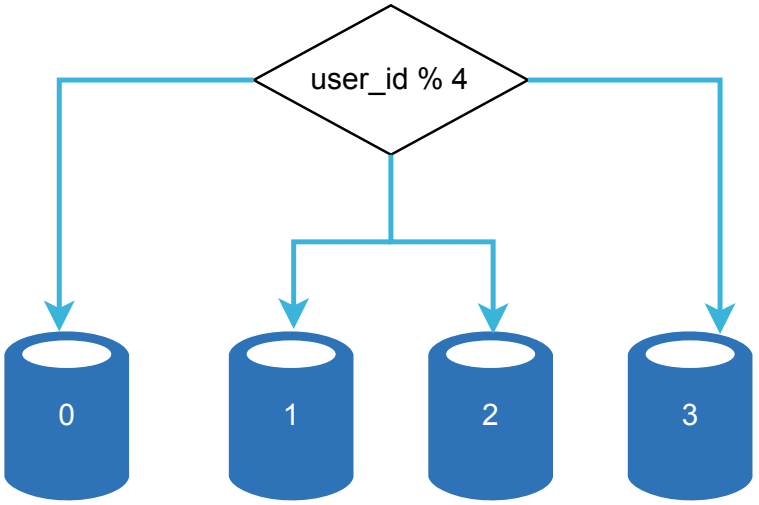
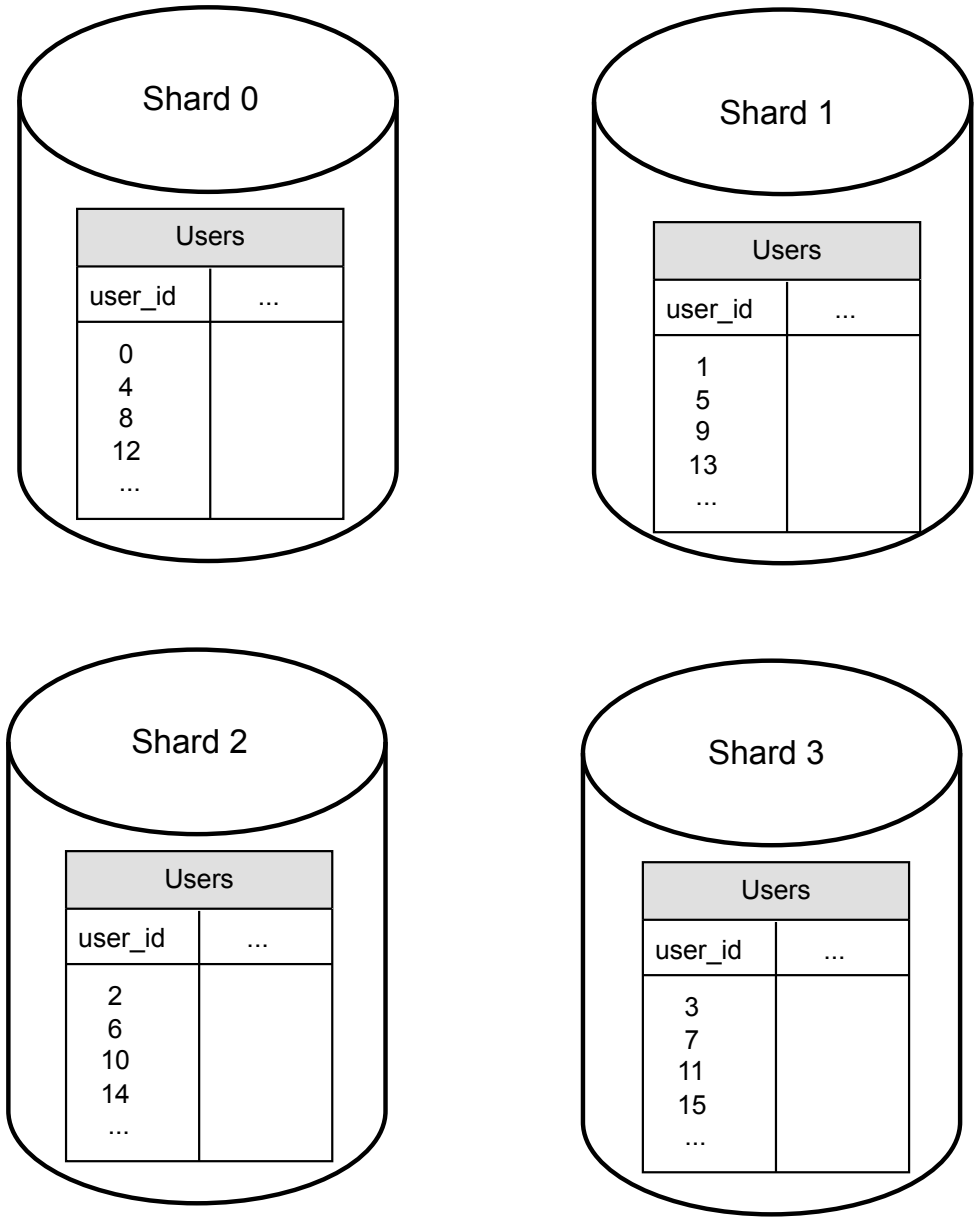


Figure 22 shows the user table in sharded databases.



The most important factor to consider when implementing a sharding strategy is the choice of the sharding key. Sharding key (known as a partition key) consists of one or more columns that determine how data is distributed. As shown in Figure 22, *"user_id"* is the sharding key. A sharding key allows you to retrieve and modify data efficiently by routing database queries to the correct database. When choosing a sharding key, one of the most important criteria is to choose a key that can evenly distributed data.

Sharding is a great technique to scale the database but it is far from a perfect solution. It introduces complexities and new challenges to the system:

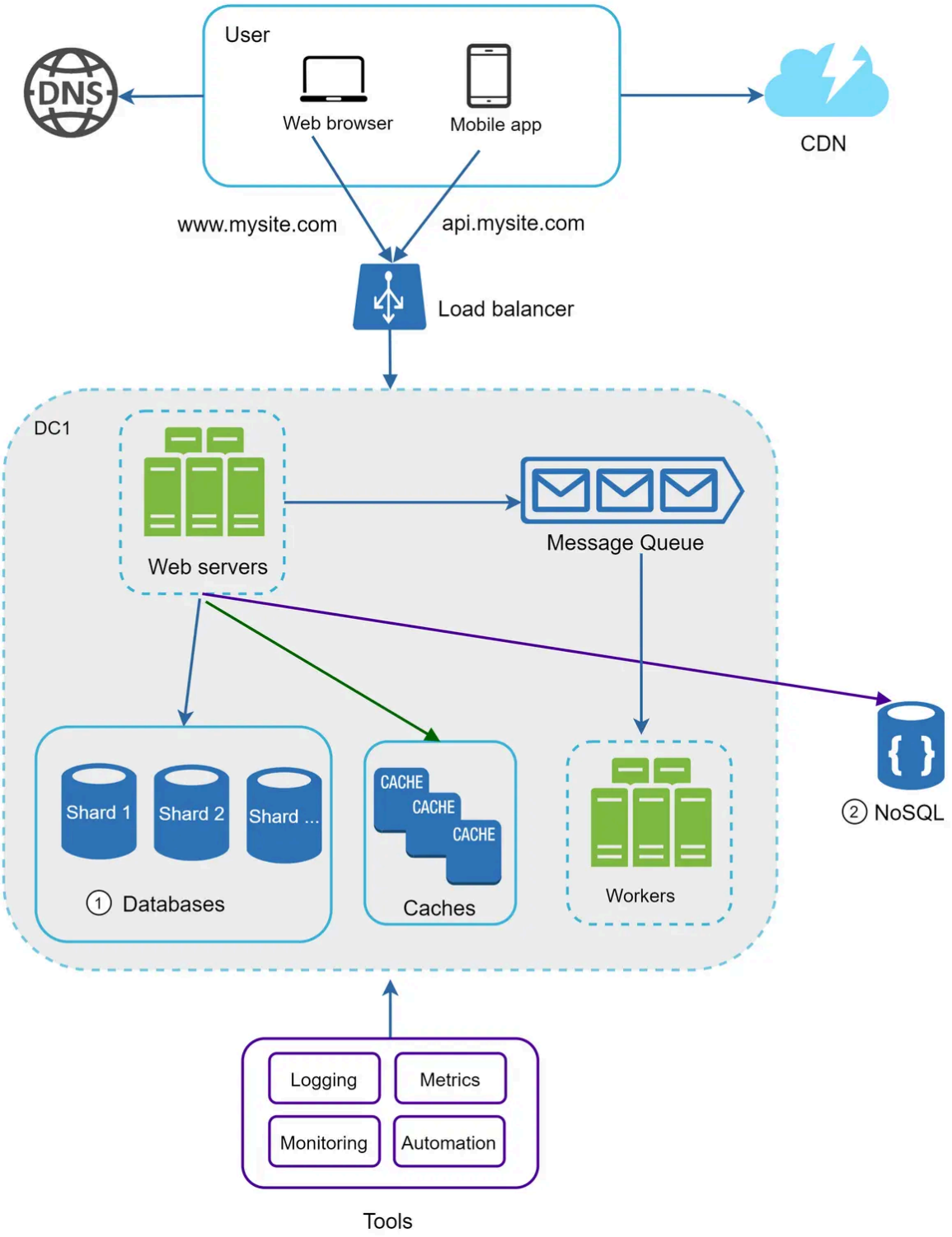
Resharding data: Resharding data is needed when 1) a single shard could no longer hold more data due to rapid growth. 2) Certain shards might experience shard exhaustion faster than others due to uneven data distribution. When shard

exhaustion happens, it requires updating the sharding function and moving data around. Consistent hashing is a commonly used technique to solve this problem.

Celebrity problem: This is also called a hotspot key problem. Excessive access to a specific shard could cause server overload. Imagine data for Katy Perry, Justin Bieber, and Lady Gaga all end up on the same shard. For social applications, that shard will be overwhelmed with read operations. To solve this problem, we may need to allocate a shard for each celebrity. Each shard might even require further partition.

Join and de-normalization: Once a database has been sharded across multiple servers, it is hard to perform join operations across database shards. A common workaround is to de-normalize the database so that queries can be performed in a single table.

In Figure 23, we shard databases to support rapidly increasing data traffic. At the same time, some of the non-relational functionalities are moved to a NoSQL data store to reduce the database load. Here is an article that covers many use cases of NoSQL [14].



Millions of users and beyond

Scaling a system is an iterative process. Iterating on what we have learned in this chapter could get us far. More fine-tuning and new strategies are needed to scale beyond millions of users. For example, you might need to optimize your system and decouple the system to even smaller services. All the techniques learned in this chapter should provide a good foundation to tackle new challenges. To conclude this chapter, we provide a summary of how we scale our system to support millions of users:

- Keep web tier stateless
- Build redundancy at every tier
- Cache data as much as you can
- Support multiple data centers
- Host static assets in CDN
- Scale your data tier by sharding
- Split tiers into individual services
- Monitor your system and use automation tools

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] Hypertext Transfer Protocol: https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

[2] Should you go Beyond Relational Databases?: https://blog.teamtreehouse.com/should-you-go-beyond-relational-databases

[3] Replication: https://en.wikipedia.org/wiki/Replication_(computing)

[4] Multi-master replication: https://en.wikipedia.org/wiki/Multi-master_replication

[5] NDB Cluster Replication: Bidirectional and Circular Replication: https://dev.mysql.com/doc/refman/8.4/en/mysql-cluster-replication-multi-source.html

[6] Caching Strategies and How to Choose the Right One: https://codeahoy.com/2017/08/11/caching-strategies-and-how-to-choose-the-right-one/

[7] R. Nishtala etc. al., "Scaling Memcache at Facebook," 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI '13): https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final170_update.pdf

[8] Single point of failure: https://en.wikipedia.org/wiki/Single_point_of_failure

[9] Amazon CloudFront Dynamic Content Delivery: https://aws.amazon.com/cloudfront/dynamic-content/

[10] Configure Sticky Sessions for Your Classic Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/classic/elb-sticky-sessions.html

[11] Active-Active for Multi-Regional Resiliency: https://netflixtechblog.com/active-active-for-multi-regional-resiliency-c47719f6685b

[12] Amazon EC2 High Memory Instances: https://aws.amazon.com/ec2/instance-types/high-memory/

[13] What it takes to run Stack Overflow: http://nickcraver.com/blog/2013/11/22/what-it-takes-to-run-stack-overflow

[14] What The Heck Are You Actually Using NoSQL For: http://highscalability.com/blog/2010/12/6/what-the-heck-are-you-actually-using-nosql-for.html

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.

(Add byte-byte go notes off 1M scale & Consistent Hashing).

Consistent Hashing :

→ It is a technique used in distributed sys & load balancing to distribute data or req. across multiple servers efficiently.

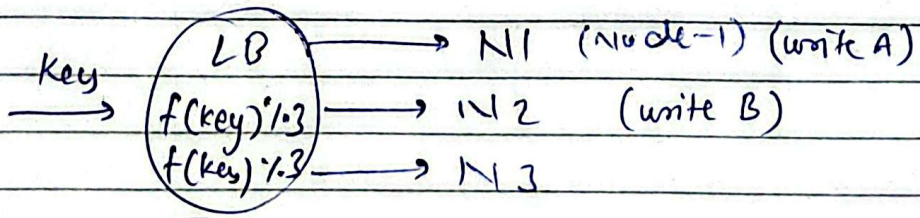
→ It reduces the amount of re-mapping or (rehashing) needed when servers are added or removed.

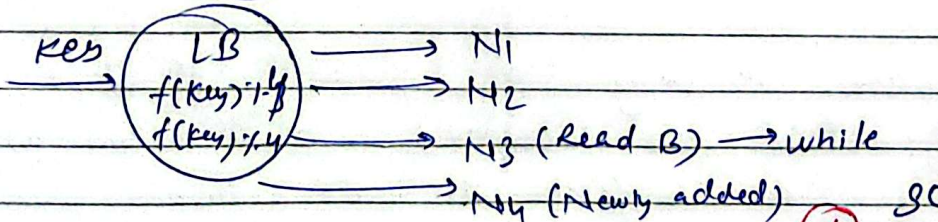
0 to $2^{160}-1$

- Servers and requests are mapped onto a virtual hash ring, where each req is assigned to the nearest server in the clockwise direction.
- When servers are added or removed, only a small portion of requests are distributed.

Note: on avg, K/N keys needed to be remapped.
K → total no. of keys, N = no. of servers/slots.

* Issues with traditional Hashing methods





go need to do
(1) rebalancing.

(2) Uneven distribution of Data → all req on 1 server.

(3) High Rehashing overhead.

(4) Scalability Problems: servers are added or removed, most of the data needs to be rehashed & re-distributed, leading to delays, inefficiency.

(5) Poor node failure handling — fail / data becomes unavail. until it is reassigned or server recovers.

* Phases / working of Consistent Hashing
  ↳ ex. SHA-1, SHA-256, MD5

Phase 1 → Hash function selection to map both keys and servers to positions on the ring.
  ↳ server.

Phase 2 → Node assignment → each key is assigned to the nearest server

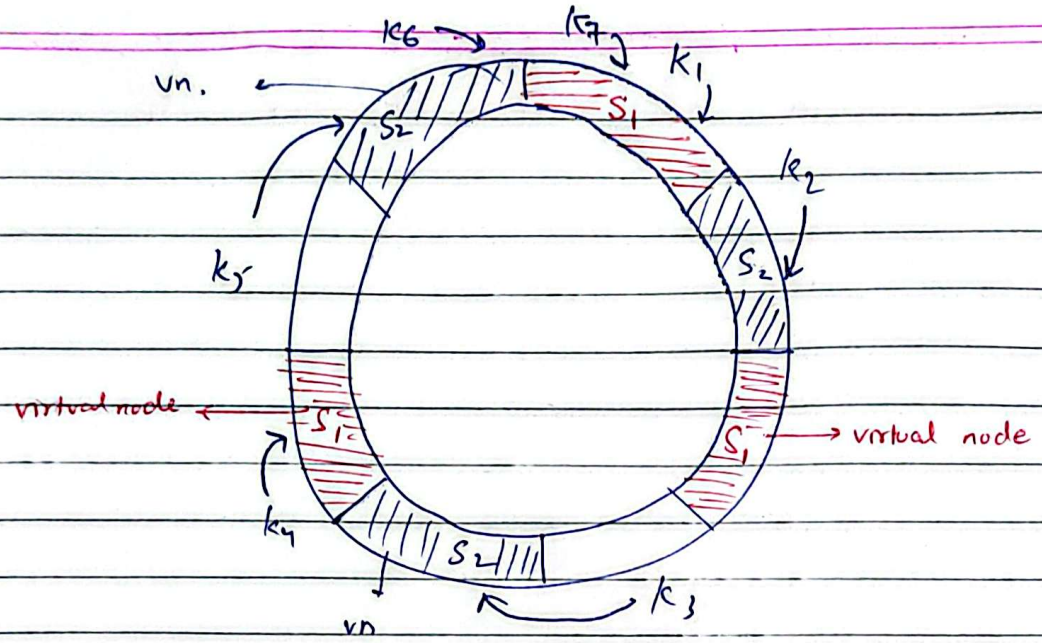
Phase 3 → Key Replication → data is replicated across multiple nodes to ensure availability in distributed sys.

Phase 4 → Node add^{n}/removal → some keys are needed to be remapped.

Phase 5 → load balancing

Phase 6 → Failure recovery → if a node fails, the keys that are assigned to it can be remapped to other nodes in the network.

CamScanner



- Virtual nodes :
  + prevents uneven data distribution.
  + Improves balancing when servers are limited
  + commonly used in

* Advantages.

(1) Load balancing
(2) Scalability: adapts easily to changes in nodes or data vol^m with little impact on performance.
(3) failure Tolerance → data remains avail. by redistributing keys when a node fails.
(4) Simplified Operation → easy to add/rem node.

* Disadvantages.

- Hash f^n complexity.
- Performance cost
- Lack of flexibility → like dynamic network cp.
- High resource use

System design of URL Shortner like

ex. Bit.ly converts long URLs into shorter

(1) Requirements → (2) Core entities → (3) API or Interface

(5) High-level Des. ← (4) Data flow

↳ (6) Deep dive.

(5) (Primary Goal) → functional req.
(6) " → Non-functional req.

- functional requirements

1.(1) Create a short url from a long url
→ optionally support custom alias
→ " " an expration time

1.(2) be redirected to the original url from the short url

- Non functional Req.

(1) low latency on redirects (~200ms)
(2) Scale to support 100M daily active user(DAU) and 1B urls.
(3) ensure uniquenes of short node.
(4) High availability [why we don't need strong consistency bcz we can wait for read after write as it's not like banking or movie ticket booking app.] [so we Need eventual consistency].

- Back of the envelope estimations [BOTE]
  - scale
  - latency
  - storage.

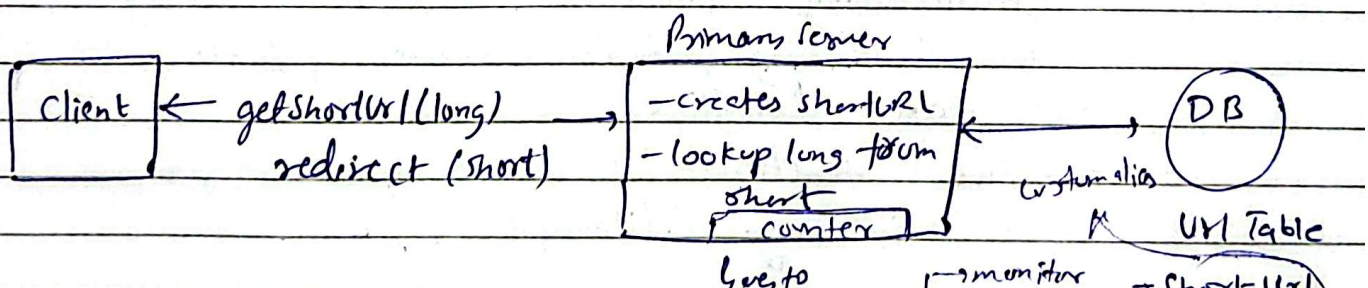
  Not doing math in interview.

(2) Core entities
- Original URL
- Short URL
- User

(3) API

// shortens a url
POST / urls ⟶ shortUrl
{
originalUrl,
alias,
expirationTime?
}

// redirection
GET {shortUrl} ⟶ Redirect to originalUrl



- 302 redirect - temporary
- 301 redirect - permanent

* It to fulfills are funct" req.

Now for non-functional ?

- fast
- unique
- short (5-7)

What can we do?

1. prefix of the long url (many common, not good)
2. random num. generator 10^9 10 chars
   - base 62 encoding, 0-9, A-Z, a-z
   - 62^6 = 56 B
   - Birthday Paradox.
     - 880k collisions (from 1B)
     - we just need to check for collision first

(Another optim Hash)

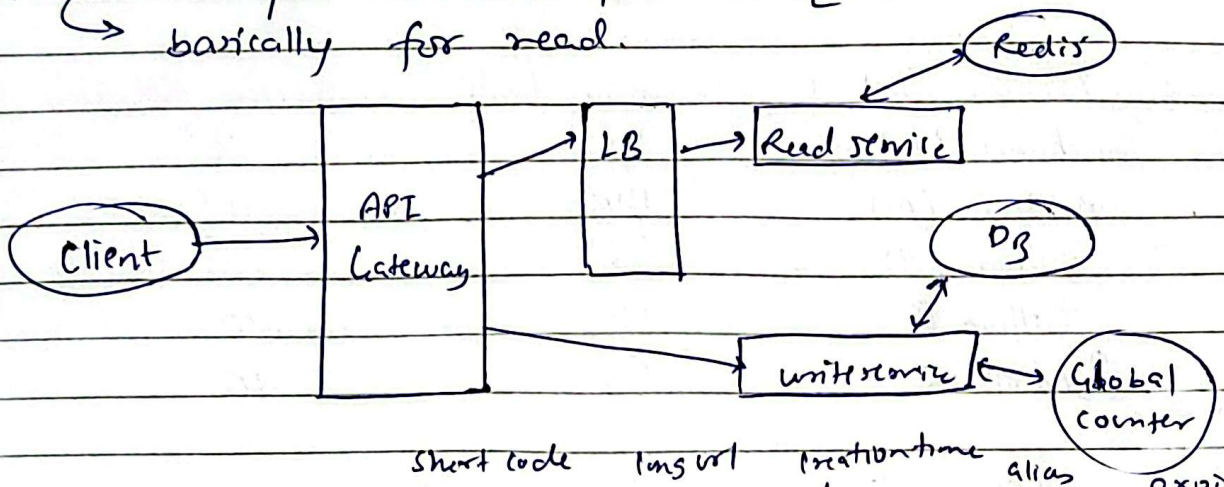
3. Hash the long url
   - md5 (longurl) → hash → base 62 (hash)[:6]
   - same collisions

4. Counter
   - incrementing a counter → base 62
   - predictability which is bad for security.
     → "warning", don't shorten private urls"
     → rate limiting.
   - bijective function. — one-one mapping
     - sqids.org. (library)

here CDN — not a good approach Similar prob like 301 redirect no tracking.

100M → 10^8, no of reqs in a day → 10^5

$10^8 / 10^5$ = 1000 rps • [10K – 100K] handle peak
↳ basically for read.



short code | long url | creation time | alias | expire

for DB: 8 bytes + 100 + 8 bytes + 100 + 8 ≈ 500

500 bytes × 1B rows ≈ 500 GB → can be handled by single instance

Note: can you use multiple instance of Cloud (AWS) of Read, write instead of of 2 service deployed — OPTIONAL.

PB → Peta byte

1KB → 2^{10} bytes → 1024 B

Source

Back of the Envelope (Calc.) (Byte Byte Go)

Number Unit	Memory Units	Specific Notation
Thousand (K)	KB	10^3
Million (M)	MB	10^6
Billion (B)	GB	10^9
Trillion	TB	10^{12}
Quadrillion	PB	10^{15}

DAU → daily active user, MAU → monthly ...
RPS → requests per second
QPS → Queries " "

let ex: Twitter 300 MAU, 10-25% make post

→ find peak hours usage or access
→ say 50% use daily = 150 M DAU
→ say 25% of DAU tweets (2 tweets)

25% * 2 = 0.5 tweets per DAU

→ estimate = post during day. = 2 x avg. (one day)

150M DAU X 0.5 tweets/DAU X 2x scaling / 86400 sec

simplify.

150M DAU . $\frac{0.5 \text{ tweets}}{\text{DAU}}$. $\frac{2x \text{ scaling}}{10^5 \text{ seconds/day}}$

$1.5 \times 10^8 \cdot 0.5 \cdot 2x \cdot 10^{-5}$

$1.5 \times 10^3 = 1500$ tweets per second

* media example. (of twitter)

150 M DAU , 10% contain pictures ①
100 KB each

1500 tweets / sec
→ 1500×10^5 1% contain videos ②
→ 150M tweets / day 100MB each

replicate for 3 copies., keep for 5 yrs

150 M tweets / day . 0.1 with pic . 100 KB ≅ 400 day / year
$(1.5 \times 10^8 \cdot 10^{-1} \cdot 10^5 \cdot 4 \cdot 10^2)$ 5 yrs . 3 copies.

$6 \times 10^{14} \times 15$

$= 9 \times 10^{15} =$ 9 PB → for images.

Storage. ~~needed~~

• for videos (from ① & ②) picture storage

Video storage $= [(1\% \times 100MB) / (10\% \times 100KB)]$

$= 100 \times 9PB$

$= 900 PB$

ex: 10^6 req. per sec , 1 server 10^3 qps.

$\Rightarrow 10^6 / 10^3 = 1000$ servers.

How Key-value stores work?

→ multiple server needed.
→ say 3 , now key 12345 %. 3 → instance
→ similar prob so need consistent hashing.
→ Now comes CAP theorem
Banks → high consistency
E-comm → high avail.

* → Now version problem may arise like two family member with same a/c add to cart but save on diff dB with diff version
sol^n ① merge dB
② ask user
③ take latest cart
sol^n vector clocks → <dB, time>

* → Now how to detect failure:
→ each server must have list of other servers
→ now that server occasionally ping or req to random no. of servers and if they don't respond., gossip spread across the cluster

Design an chat Server (Messenger/whatsapp)

→ user should be able to send and receive message from friends and groups in near real time.

① Functional Req.
(1) user should be able to reg/login
(2) one-to-one messaging
(3) Group messaging
(4) support both Text/Media message.
(5) Message history
(6) Delivery / Read receipts

② Non-funct. Req.
(1) Scale: 1B users, 100 msgs/day → 100B msg/day
⇒ if one media/text 1Kb ⇒ 1KB x 100B = 100 TB/day

(2) CAP: need → highly available, eventual consistency
(3) low-latency <300 ms
(4) Highly reliable with no message loss.

② Core Entities.
(1) User
(2) Group
(3) Message/chat

③ API design.
① POST /v1/user/register { forBody: userName, email/phone..}

② PUT, etc..

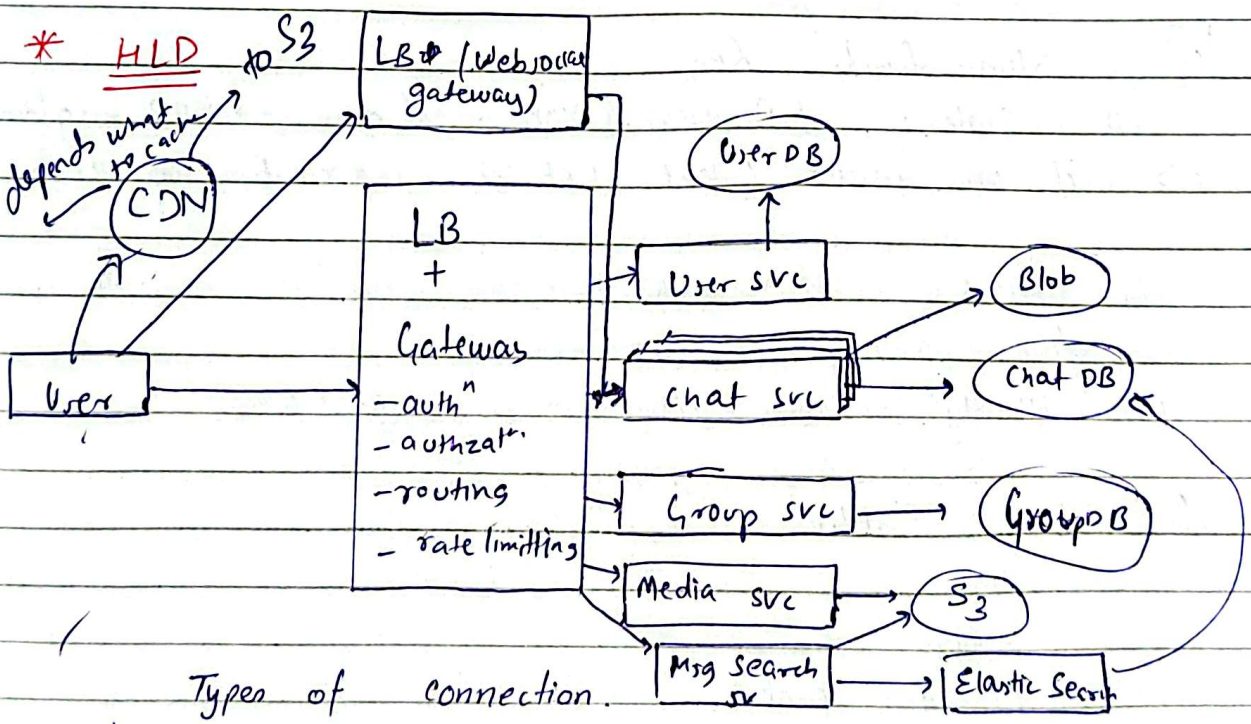
one-to-one:

1. WS : /v1/ messages / send. → handle blue ticks (for convo)
2. GET /v1/ chat / {userId} : List<Chat> : Pagination
3. GET /v1/ message / {userId} / {receiverId}

Group-msg

1. POST /v1/ groups / create
2. POST /v1/ groups/ {groupId} / add
3. POST /v1/ groups/ // / remove
4. WS: /v1/ messages / send → handle blue ticks
5. GET /v1/ groups / {groupId} / message : Lazy load

* HLD

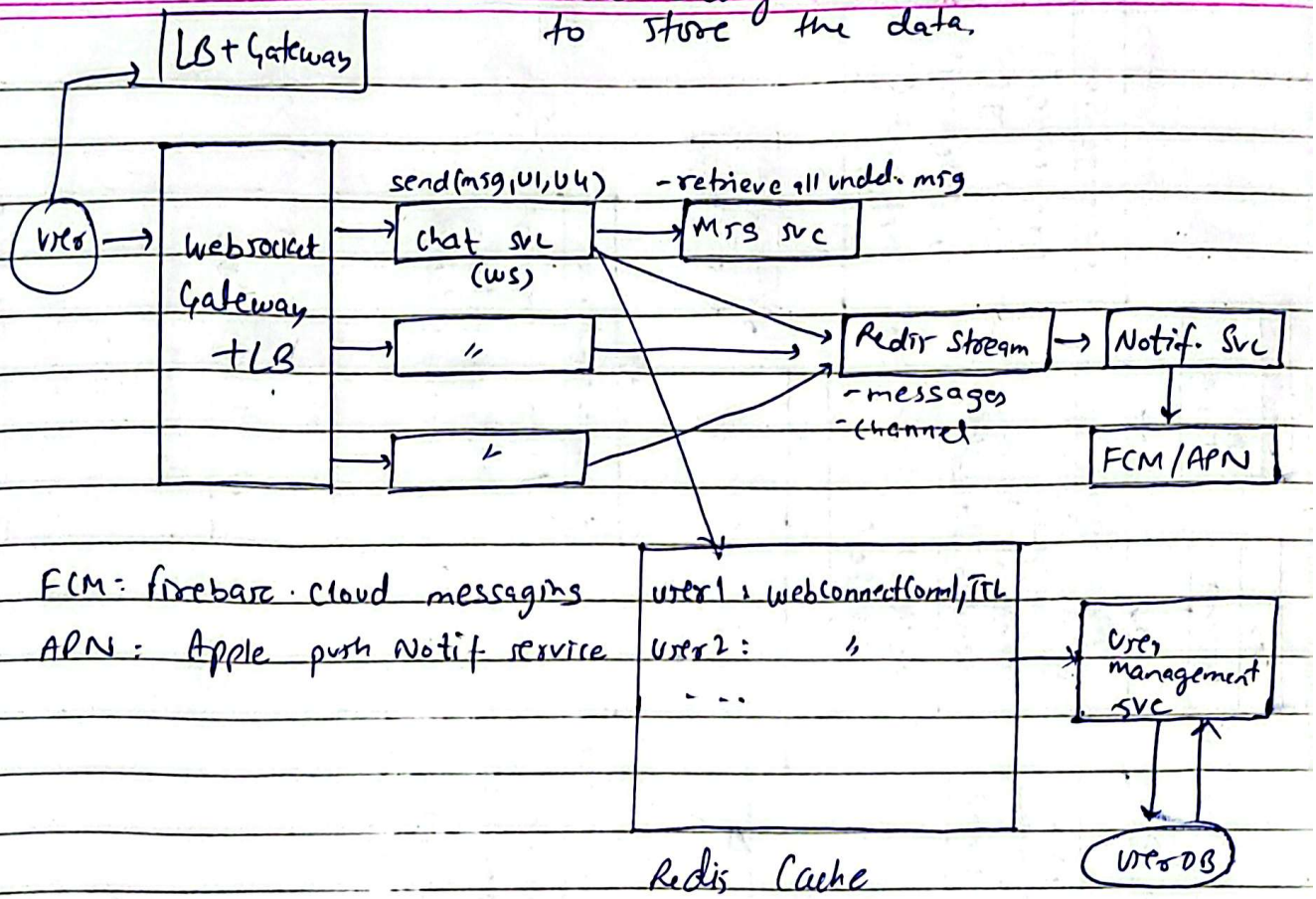


Types of connection.

1. HTTP
2. Long polling → poll to server for some time
3. Server Sent Event (SSE) → on event server → client
4. Websocket.

Note: Since our chat service is huge & highly dependend on the WS so we have to use a dedicated LB for that.

local storage on device can be used to store the data.



FCM: firebase cloud messaging
APN: Apple push Notif service

① Association => "——→" meaning one class uses / knows about another class.

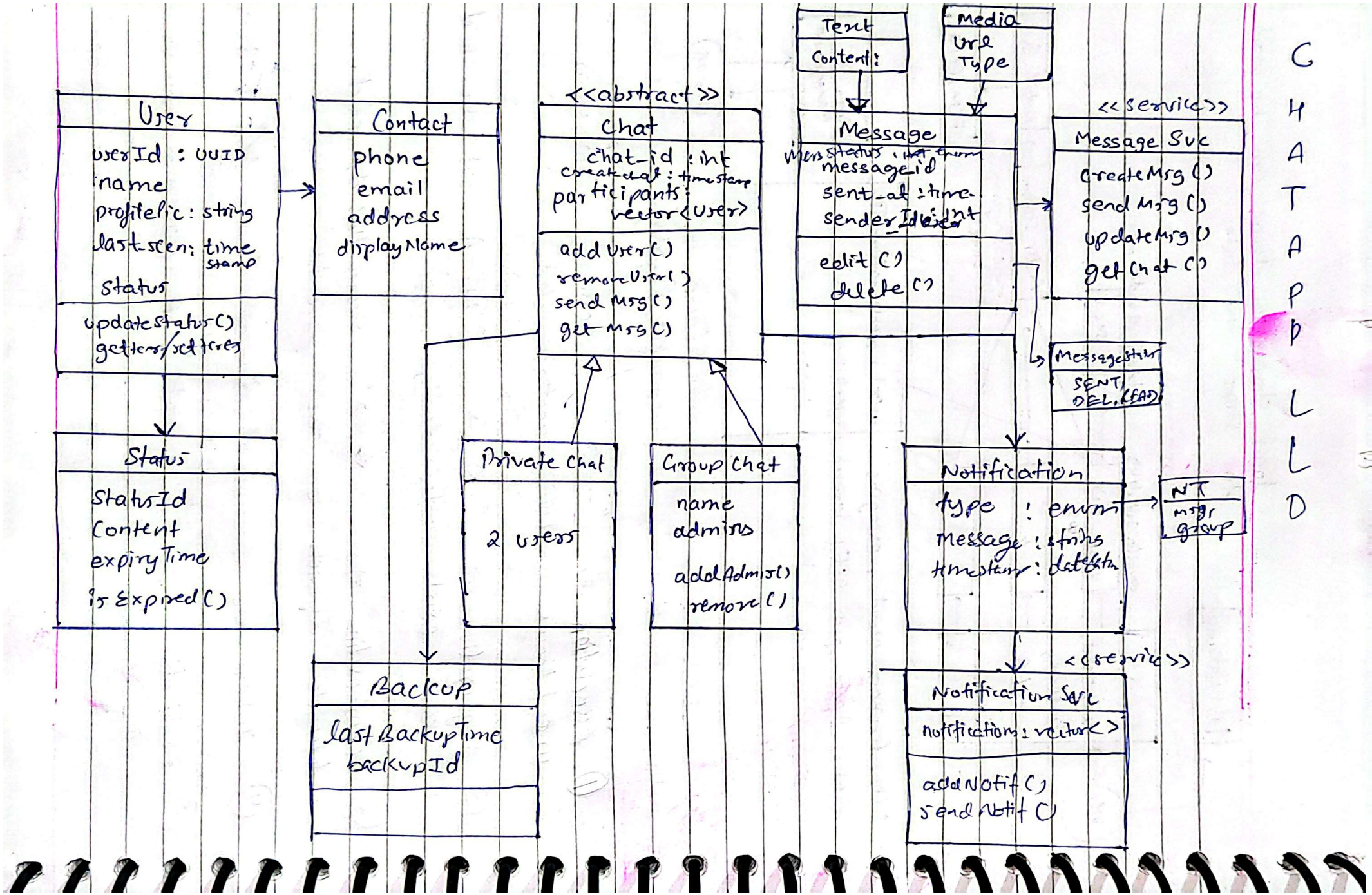
② Aggregation => "◇——→" HAS-A, but the child can exist independently

③ Composition => "◆——→" strong HAS-A, child's lifecycle depends on the parent.

④ Inheritance / Generalization => "——▷" IS-A

⑤ Realization => "- - -▷" A class implements an interface

paymentService - - -▷ payment


CHATAPP LLD
User
userId : UUID
name
profilePic : string
last_seen : time stamp
status
updateStatus()
getters/setters
Contact
phone
email
address
displayName
<<abstract>>
Chat
chat_id : int
createdAt : timestamp
participants : vector<User>
addUser()
removeUser()
sendMsg()
getMsg()
Text
Content
Media
Url
Type
Message
messageId
sent_at : time
senderId : int
edit()
delete()
<<service>>
MessageSvc
createMsg()
sendMsg()
updateMsg()
getChat()
MessageStatus
SENT, DEL, READ
Status
StatusId
Content
expiryTime
isExpired()
Private Chat
2 users
Group Chat
name
admins
addAdmin()
remove()
Notification
type : enum
Message : string
timestamp : datetime
NT
msgr
group
<<service>>
NotificationSvc
notifications : vector<>
addNotif()
sendNotif()
Backup
lastBackupTime
backupId

Design a distributed Rate Limiter

Sys Interface

(1) Req → (2) Core Entities → (3) API → (4) Data flow

(6) Deep Dive ← (5) HLD ←

(1.1) Functional Requirements.

- identify users by id, IP, or api key
- limit requests based on configurable rules
- return proper ~~head~~ error headers and ~~stat~~ status codes.

Scale

- 100 M DAU
- 1M rps

(1.2) Non-functional requirements.

- availability >> consistency
- low latency rate limit checks (<10 ms)
- scalable to 1M rps

(2) Core Entities

- Request
- client (ID, user ID, Api key)
- Rules (100 r/s — per user)

(3) System Interface

isRequestAllowed (clientId, rulesId) → { passes: boolean, remaining: number, resetTime: timestamp }

msvc → microservice

Q.1 where should our rate limiter live in our architecture?

Q.2 How should we identify clients?

Ans1 Options to place: (1) within each microservice – fast but if user make 2 req for msvc 1 and msvc 2 each ratelimiter will assume it as one.

(2) ratelimiter as own service solves above prob in (1) but introduces latency (and we don't want that).

(3) Place in the [LB + Gateway] – solves all above prob global knowledge + latency.

Ans2 Ask (req) → all users authenticated, ii) developer api, need api key, as anonymous may have lower rate limits for api call.

Note: There are 4 commonly known Rate limiting algos:

(1) Fixed window counter algo.
(2) Sliding window log
(3) Leaky bucket algo
(4) Token Bucket algo.

Optimized version of (2) → Sliding window Counter

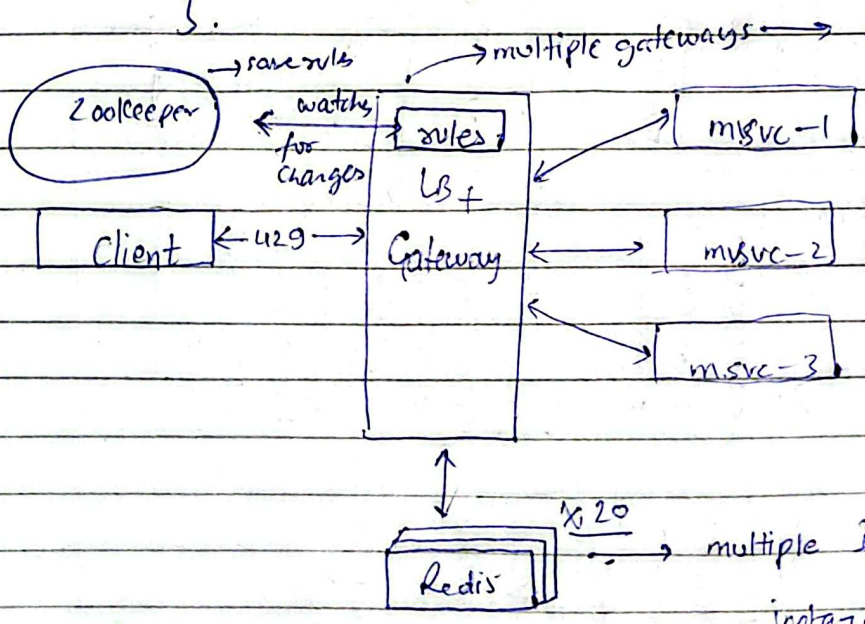
(Notes will be added in PDF)

③ API design.

- Admin (Create rules) : POST /v1/admin/rules
- Admin GET /v1/admin/rules ? clientId = xyz (csv)
- Admin GET /v1/admin/rate-limiter/metrics ? resource= {url}

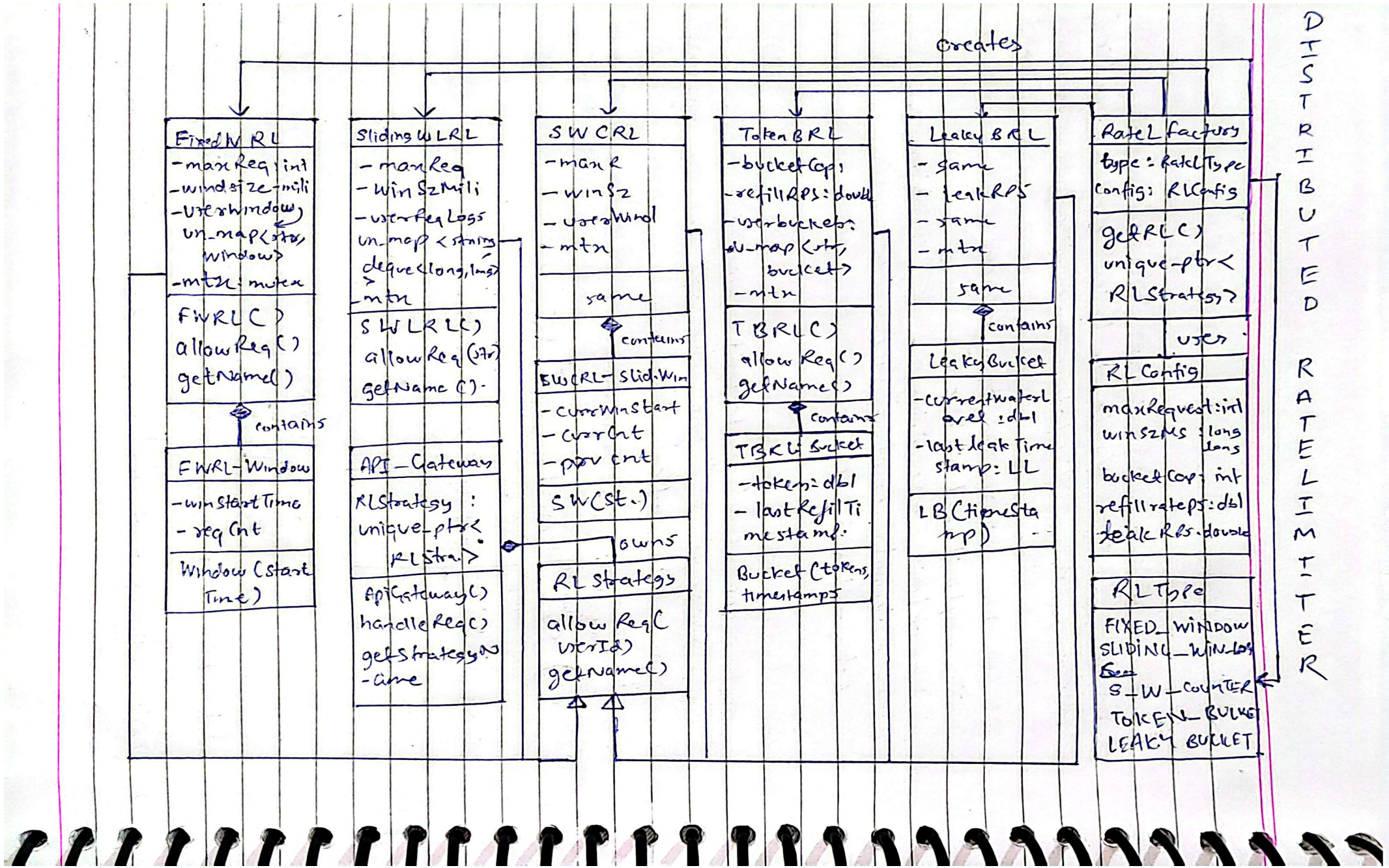
HTTP 1.1 429 Too Many Requests
X-RateLimit-Limit : 100
X-RateLimit-Remaining : 0 ↱ unix timestamp
X-RateLimit-Reset : 1712345708
Retry-After : 30

{
"error" : "rate_limit_exceeded"
"message" : "you have exceeded 100 rpm/min. Retry after 30 sec"
}.



let 50K req per redis instance 1M/50K = 20

redis cluster → distributes req across instances.


DISTRIBUTED RATELIMITER
creates
FixedWRL
-maxReq: int
-windsize-mili
-UserWindow)
un_map<str, window>
-mtx: mutex
FWRL()
allowReq()
getName()
contains
FWRL-Window
-winStartTime
-reqCnt
Window(startTime)
SlidingWLRL
-maxReq
-WinSzMili
-userReqLogs
un_map<string, deque<long, long>>
-mtx
SWLRL()
allowReq(str)
getName()-
API-Gateway
RLStrategy: uniquePtr<RLStra.>
APIGateway()
handleReq()
getStrategy()
-ame
SWCRL
-maxR
-winSz
-userWind
-mtx
same
contains
SWCRL-SlidWin
-currWinStart
-currCnt
-prevCnt
SW(st.)
owns
RLStrategy
allowReq(UserId)
getName()
TokenBRL
-bucketCap:
-refillRPS: double
-userbuckets un_map<str, bucket>
-mtx
TBRL()
allowReq()
getName()
contains
TBRLiBucket
-token: dbl
-lastRefilTimestamp:
Bucket(tokens, timestamps
LeakyBRL
-same
-leakRPS
-same
-mtx
same
contains
LeakyBucket
-currentwaterlevel: dbl
-lastleakTimestamp: LL
LBC(tipneStamp)
RateLfactory
type: RateLType
config: RLConfig
getRLC()
unique_ptr<RLStrategy>
user
RLConfig
maxRequest: int
winSzMS: long long
bucketCap: int
refillrateps: dbl
leakRPS: double
RLType
FIXED_WINDOW
SLIDING_WINLOG
S_W_COUNTER
TOKEN_BULKET
LEAKY_BUCKET

Building Idempotent APIs:

(1) What is Idempotency!

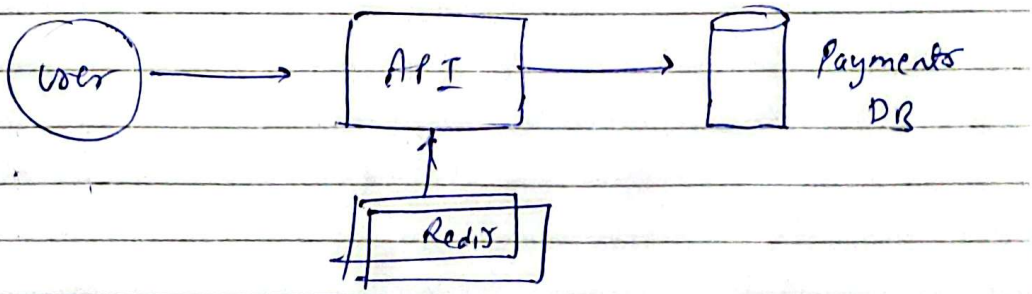
→ An API is idempotent if making the same request multiple times produces the same final result as making it once.

Ex: POST /payments
Idempotency-Key : abc123
{
"amoun" : 500, → server for process payment
"userId" : 101,
}

if the client sends the exact same request again with the same key → the server should not create other payment.

(2) - 3 conditions of failure where it needed.

(1) request failed before reaching to server.
(2) " " while server is executing.
(3) " " after execution done at server and resp was sending to client.



Simple key-value store. with TTL

(3) How to make Idempotent API?

→ client generates a unique key
Idempotency-key : hash of key (value)

→ this stores in DB

→ similar to cache hit - miss.

→ if req. comes first time cache the key

→ next time show the prev state (customize)

→ if req completes successfully delete the key.

(4) Problem: what if multiple req of same logic req. comes simultaneously.

→ sol^n use (UNIQUE) to create the idempotency table → for idempotency key

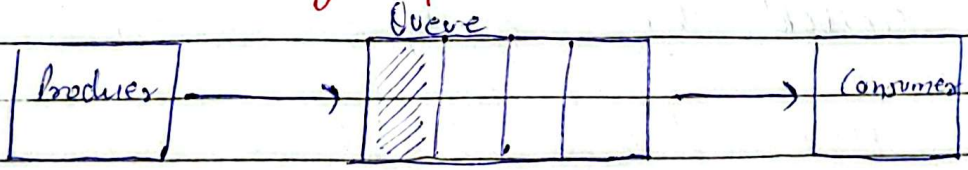
Design Patterns for High Availability.

	Availability	Downtime / yr
Novice	99 %	3.65 days
Small Scale	99.9 %	8.77 hrs
Medium "	99.99 %	52.60 mins
Google Amazon Meta Large "	99.999 %	5.26 min
Large "	99.9999 %	3156 sec
Rare use case (medical GPS)	99.99999 %	3.16 sec
Rare use case (medical GPS)	99.999999 %	315.58 milli sec

Explain everything, LB, Replicating, sharding, Cache, Indexing etc.

Message Queue ⟹ kafka / RabbitMQ

What is message queue and Need?



→ A messaging queue is a sys that allows one application to send message to another asynchronously

→ adv : servie/server doesn't necessarily need to wait for the payment service to finish.

→ adv : retry capability when server fails.

adv → Pace matching

Queue will help here.

A → 15 msg/s
B → 10 msg/s
C → 15 msgs

Server can process only 15 msg/s

adv → decoupling: Prod. & Consu. don't need to know much about each other.

Point to Point : Queue based Messaging.

→ one message is intended to be processed by one consumer.

→ if multiple consumers are there then one msg goes only to one consumer.

Q. Pub-Sub: one published message can be received by multiple subscribers.

Q. How does a Msg Queue work ?

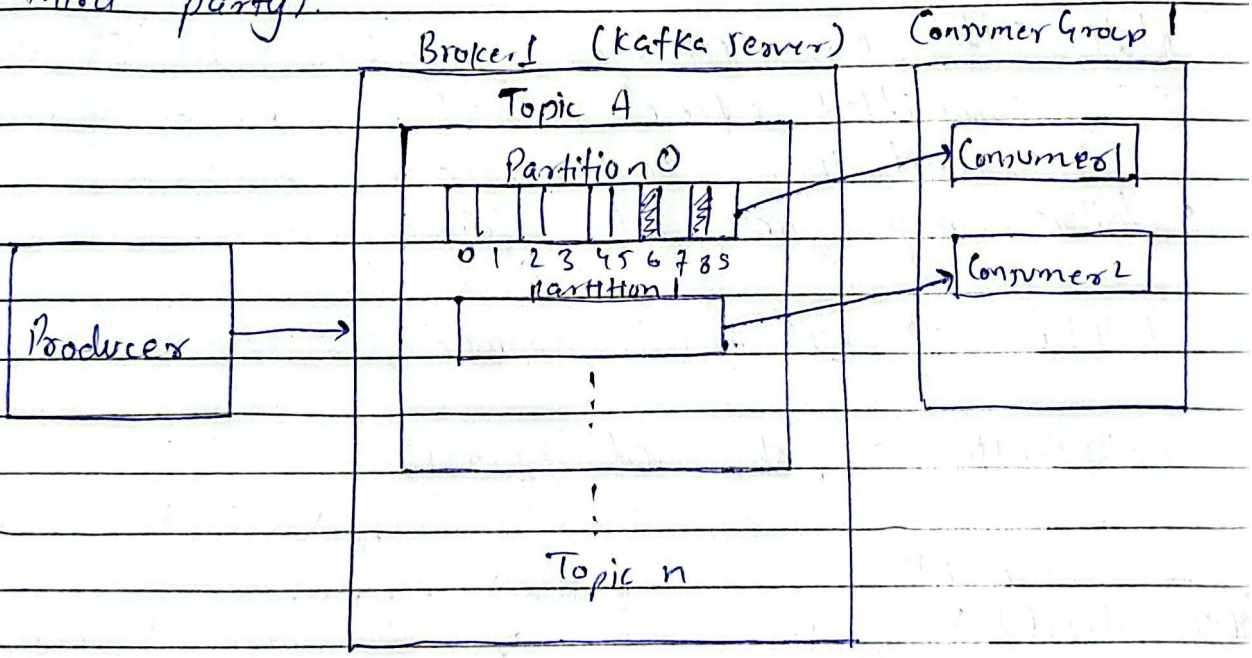
→ Components

1. Producer → application that publish messages.
2. Consumer → " that consumes/process "
3. Consumer Group → work together to consume a topic
4. Topic → is logical category/feed where kafka stores related events.
5. Partition → kafka topic is divided in partitions
   ↳ provide parallelism
6. offset → every msg in a partition has an offset it tells where is the message located in the partition. [committed offset → 4] → means till 4 all msg are read.

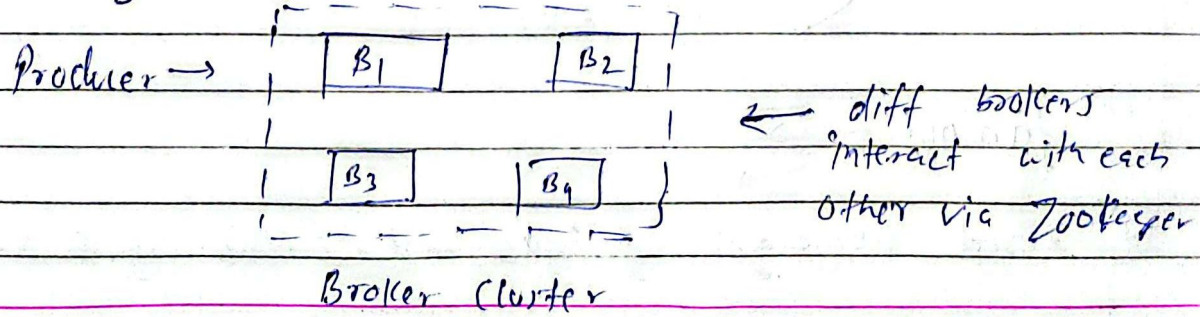
Replicas → followers] Partition.
Not Replicas → Reader]

(7) Broker → Kafka server → stores & serves kafka data.
(8) Kafka Cluster → a collection of kafka brokers working together.
(8) *Replication → kafka can maintain multiple copies of a partition.., leader falls follower takes its place
(9) Zookeeper → for cluster coordination & metadata mangement

Note: fully removed in Apache Kafka 4.0 now it uses KRaft (bcz Zookeeper was third party).



one partition can be shared to many CG but one partition cannot be shared to many C within the same CG



MSG:

Two not mandatory
- Key :
- * value : actual MSG
- Partition

imp mand.
- * Topic : Topic name

when new msg comes → checks for key
- not found ← partition ← not found
  - then follows Round Robin (RR) fashion to store msg.

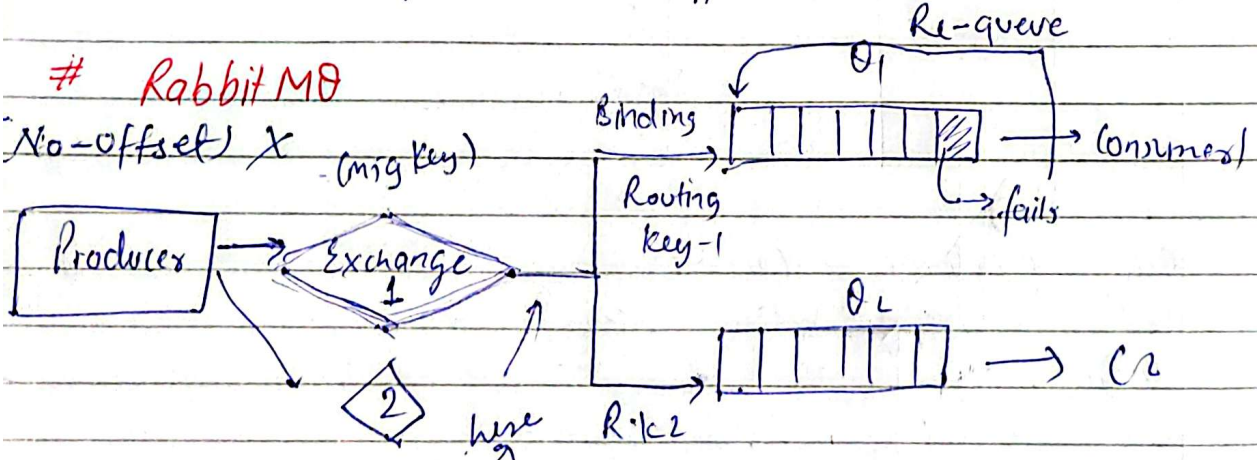
Note: • "if" one consumer goes down in CG then committed offset used by another consumer to read the last processed msg's.
• failure msg goes into failure queue.

KAFKA → pull based approach → consumer pulls/poling

RABBITMQ → push based approach

RabbitMQ

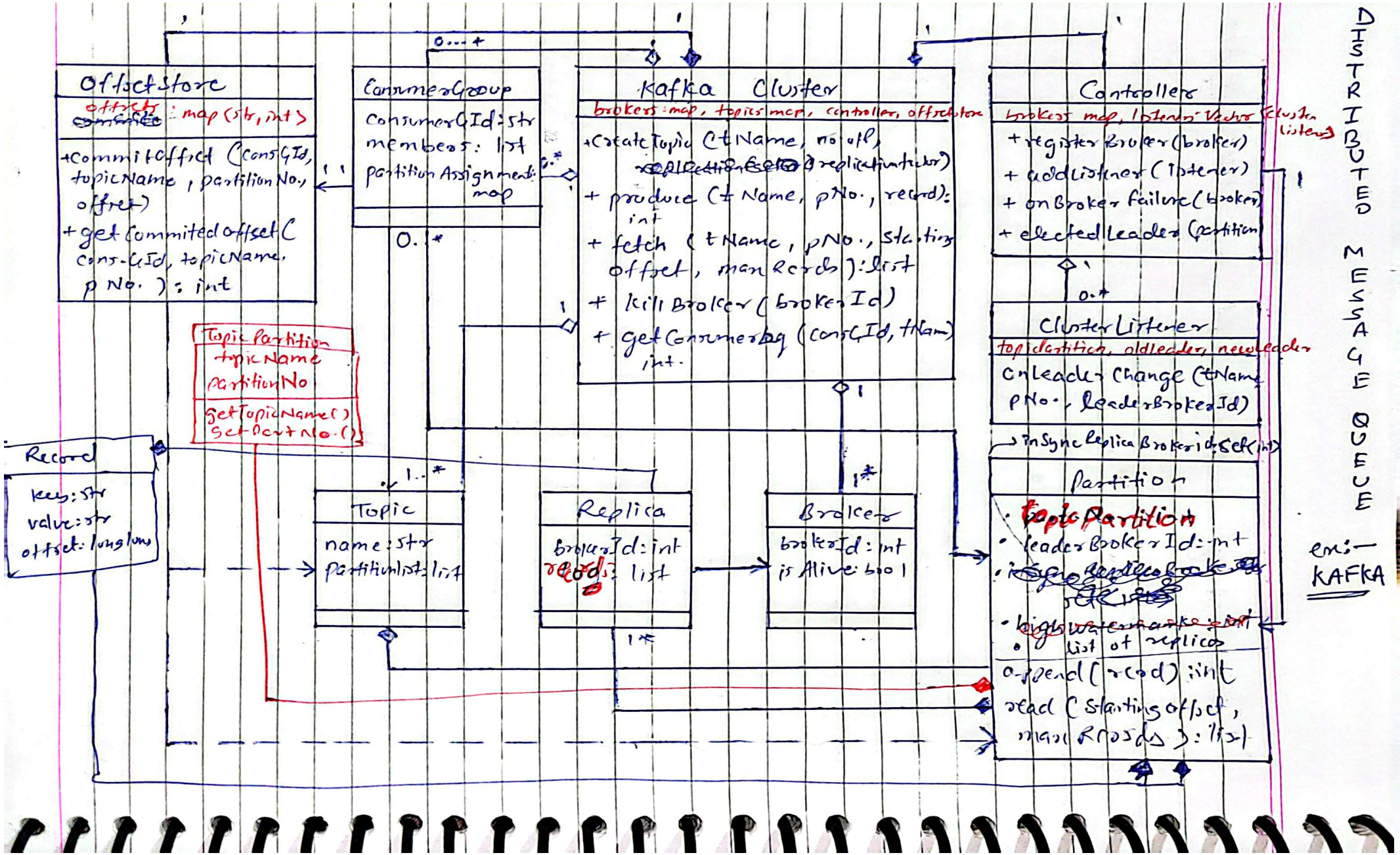
No-offset X



① fan out

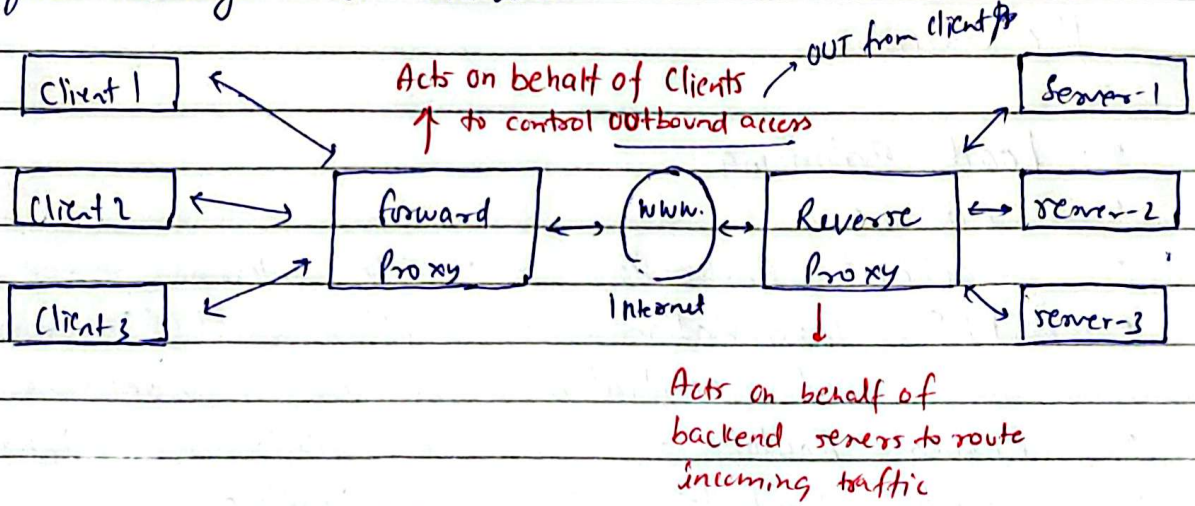
② Direct Exchange [Exact match MSG Key & RK

③ Topic Exchange → wild card ⇒ *123 = AB_123


DISTRIBUTED MESSAGE QUEUE
eg:- KAFKA
OffsetStore
offsets : map (str, int)
+CommitOffset (consGId, topicName, partitionNo., offset)
+get Commited offset (consGId, topicName, pNo.) : int
ConsumerGroup
consumerGId: str
members: list
partitionAssignment: map
Kafka Cluster
brokers: map, topics: map, controller, offsetStore
+CreateTopic (tName, no of P, replicationFactor)
+ produce (tName, pNo., record): int
+ fetch (tName, pNo., starting offset, max Records): list
+ killBroker (brokerId)
+ getConsumerGrp (consGId, tName) int.
Controller
brokers: map, listeners: Vector (cluster listeners)
+ registerBroker (broker)
+ addListener (listener)
+ onBrokerFailure (broker)
+ electedLeader (partition)
ClusterListener
topicPartition, oldLeader, newLeader
OnLeaderChange (tName, pNo., leaderBrokerId)
TopicPartition
topicName
partitionNo
getTopicName()
getPartNo.()
Record
key: str
value: str
offset: long long
Topic
name: str
partitionList: list
Replica
brokerId: int
record: list
Broker
brokerId: int
isAlive: bool
inSyncReplicaBrokerIdSet (int)
Partition
topicPartition
leaderBrokerId: int
inSyncReplicaBrokerIdSet
highWatermarkOffset
list of replicas
append (record): int
read (startingOffset, maxRecords): list

Forward Proxy v/s Reverse Proxy

Proxies act as intermediaries b/w client & servers and help manage how requests and responses flow through a network.



① Forward Proxy :

- sits b/w clients and external servers.
- destination server generally communicates with the proxy rather than directly with the client.

Use cases :

- Client IP Masking (although not guaranteed)
- Access control (like tiktok banned)
- Content filtering
- Caching
- Monitoring & Logging
- Controlled access to restricted resources

→ A FP may be explicitly configured on the client, but transparent or intercepting forward proxies can also operate without explicit client-side proxy config.

(2) Reverse Proxy ex: Load Balancer.

- sits in front of one or more backend servers and accepts request on their behalf.

Use Cases:

- Load Balancing
- Caching
- Backend Abstraction → Hide internal server add.
- TLS Termination → Handles HTTPS connections and centralize certificate management.
- Request Routing → acc to hostname, URL, headers
- Security Controls → rate limitting.

CamScanner

Load Balancing and its algos:

→ A Load Balancer is a software or hardware device that keeps any one server from becoming overloaded.

Two primary approaches:-

① Dynamic Load Balancing

→ It condsider the state of the server must communicate with servers, more efficient load balancing.
example:
(1) Least Connection Method
(2) Weighted Least Connections Method
(3) Least Response Time
(4) Resource Based

② Static Load Balancing

→ Ignores the state of any particular server simpler and more efficient to implement
examples:
(1) Round Robin (RR)
(2) weighted RR
(3) Source IP Hash
(4) URL Hash
(5) Randomized Algos.

Static

① RR

→ distributes traffic to a list of servers in rotation using the DNS.

→ An authoritative nameserver will have a list of diff 'A' records for a domain and provides a diff one in response to each DNS query.

A record → IP DNS details.

→ Req are assigned to servers one by one in a circular manner.

Pros: easy to implement, minimal config.

Cons: ignores server load, ex: Server-1 => CPU = 4, Mem: 8Gb, Server-2 => CPU=2, Mem 4Gb both get equal load. no Good ✗ as S-1 can handle much more.

② W-RR

– distributes req based on assigned weight values that represents each server's capacity.

Weight ↑ => Req ↑

Pros: prevent weaker servers from overload.

Cons: still static cannot adapt to suddent runtime load changes.

* DNS-RR by IP add. as server.

→ DNS + client caching.

→ No health checks.

③ Source IP Hash :

→ server selected by computing hash of client's source IP

→ Req from same IP goes only to same server while server pool (same set) and hashing config remain unchanged.

Pros : Provides a form of session Affinity without requiring a separate session cookie.

Cons : Affinity is not guaranteed if the backend pool or config changes, and multiple users behind the same public IP may be mapped to the same server.

Affinity → Keeping a client connected to same server

④ URL Hash

Servers

① → ex.com/payments → Payments

② → ex.com/login → Login

hash table 0: — ①

1: — ②

⑤ Random Algo

→ as name.

→ best for clusters consisting of nodes with a similar specs.

(3) Weighted - LCM

Dynamic

(1) Least Connection Method (LCM)

→ routes new requests to the server with the fewest active connections.
→ focuses on balancing workload by considering the current load on each server.

Pros: effective load balance.

Cons: more resource intensive due to real-time load eval,

(2) Least Response Time Met:-

→ uses current or recent response-time measurements rather than relying only on historical data.

→ Continuous updates routing decisions as server performance changes.

Pros: reduce latency
Cons: requires continuous monitoring and additional runtime metrics.

(4) Resource Based

→ Routes requests to the server with the most available resources at that moment
→ prevents server overload by continuously monitoring resource usage.

Pros: optimizes resource utilization by considering CPU, memory and bandwidth in real-time.

Cons: More complex to implement and may require frequent resource checks.

(5) Geolocation Based.

→ server closest to client over Globe.

Cloudflare uses ⇒ global server load balancing (GSLB).

— Establish TCP Connection →

— " TLS Connection →

— HTTP Req →
msg: encrypted

www. Server

← HTTP Res —

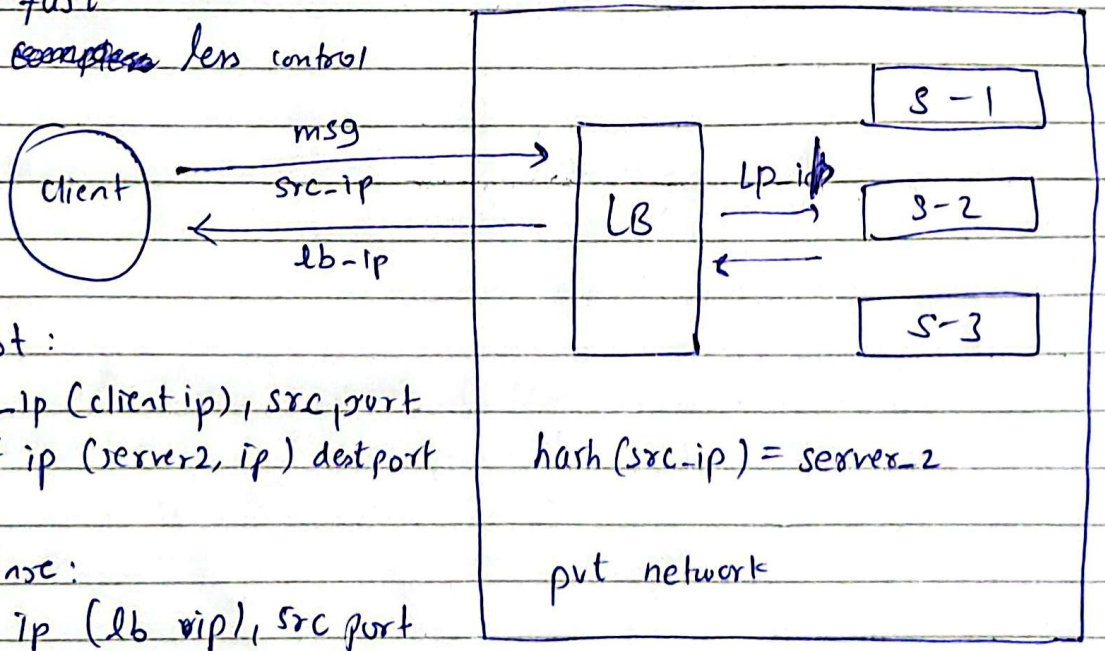
← closes TLS connection —

← closes TCP connection —

LB can terminate any one or both TCP, TLS connection.

L4 (NAT) Mode) — behaves like NAT so NAT.

- fast
- ~~complex~~ less control

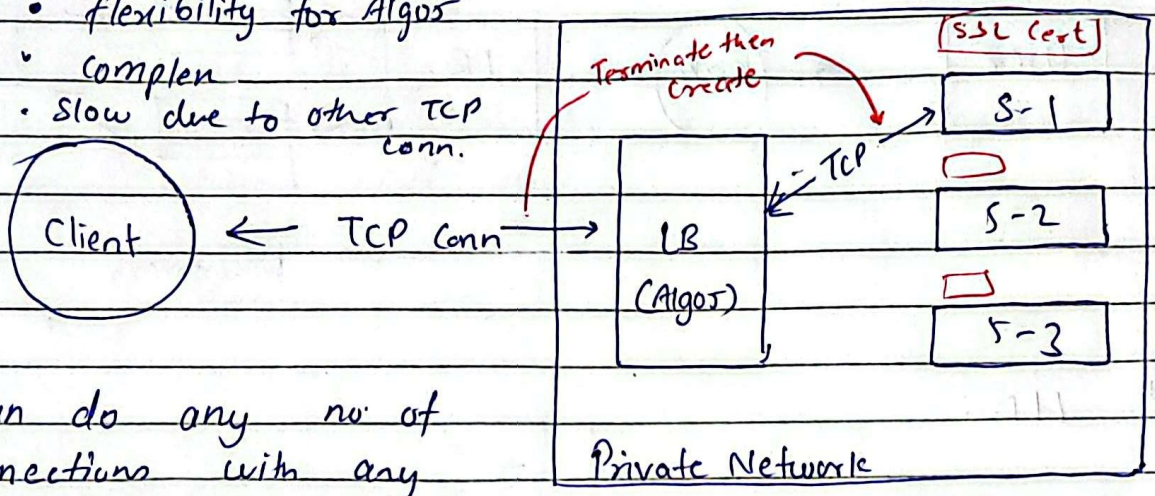


Request:
src_ip (client ip), src port
dest ip (server2, ip) dest port

Response:
src ip (lb vip), src port
dest ip (client ip), dest port.

L4 (Proxy Mode)

- flexibility for Algos
- complen
- slow due to other TCP conn.

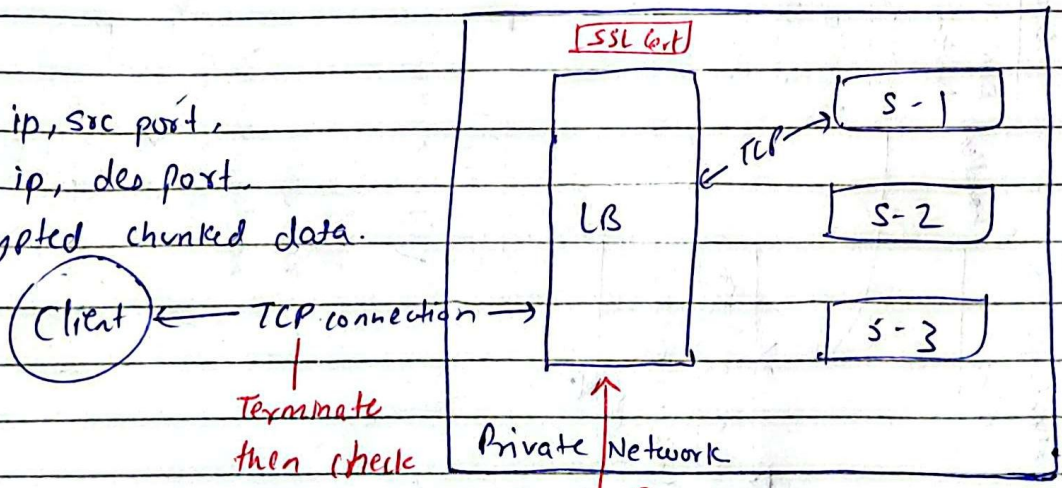


LB can do any no of connections with any server based on the algorithm.

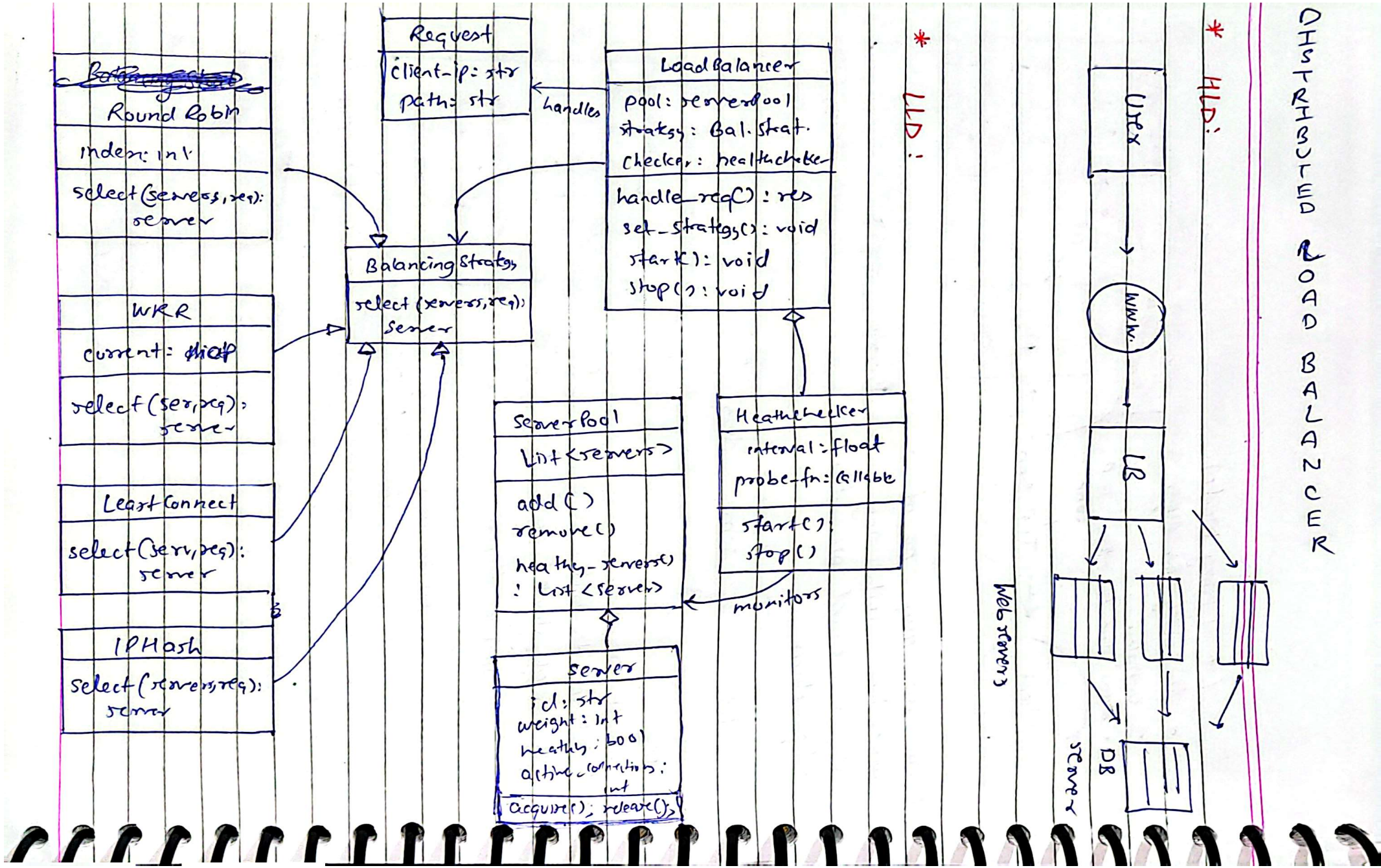
server decrypt data

L7 (same diag ↑)

Req:
src ip, src port,
dest ip, des port.
encrypted chunked data.



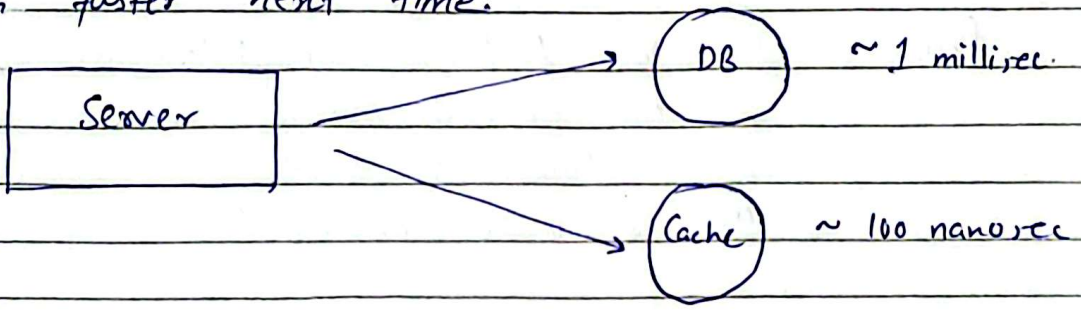
Terminate then check acc to HTTP Req (where to go /pay, /login)


DISTRIBUTED LOAD BALANCER
* HLD:
* LLD :-
User
www.
LB
DB server
Web servers
Request
client_ip: str
path: str
handles
LoadBalancer
pool: serverPool
strategy: Bal. Strat.
checker: healthchecker
handle_req() : res
set_strategy() : void
start() : void
stop() : void
BalancingStrategy
select (servers, req): server
Round Robin
index: int
select(servers, req): server
WRR
current: dict
select(ser, req): server
LeastConnect
select(serv, req): server
IPHash
select(servers, req): server
ServerPool
List<servers>
add()
remove()
healthy_servers() : List<Server>
HealthChecker
interval: float
probe_fn: Callable
start()
stop()
monitors
Server
id: str
weight: int
healthy: bool
active_connections: int
acquire(); release()

Caching

Q. what is Caching ?

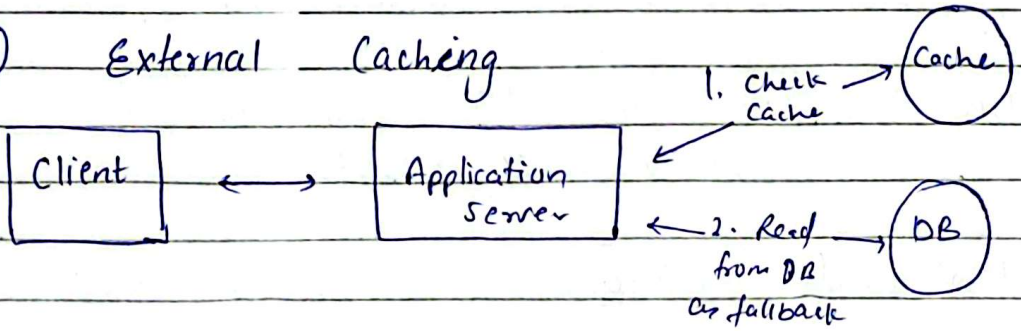
→ A cache is just a temporary storage that keeps recently used data handy so you can get it much faster next time.



memory (RAM) is roughly 10,000 x faster than disk (SSD) on avg.

Q. Where to Cache ?

① External Caching



Pros: 1. low latency
2. Reduce DB load
3. More cache capacity
4. Survive app-server restarts

Cons: 1. Extra network hop
2. Cache miss penalty
3. Cache invalidation problem [DB updated, cache not]
4. Cache failure

(2) In-process Caching

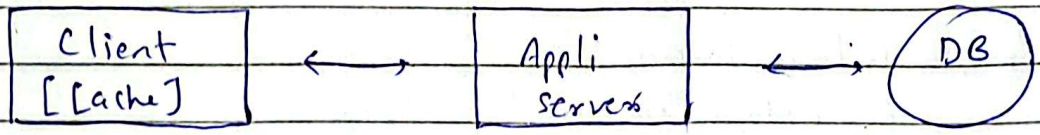
Pros:
1. Very fast - low latency
2. No external infra
3. Reduces DB load
4. Simple to implement

Cons:
1. Separate cache for each server → not share data automatically.
2. Cache inconsistency across-servers
3. Cache lost when server restarts
4. Consume application RAM.

(3) CDN

~ same like external more overhead, more traffic handles

(4) Client - Side Caching



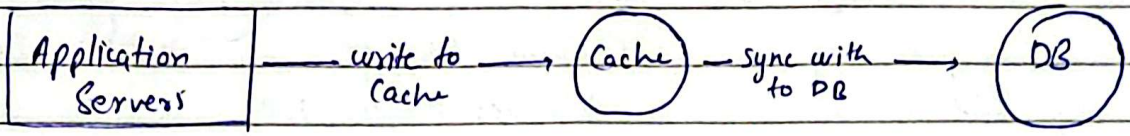
Pros
1. Work with poor connectivity
2. Fastest resp.
3. Reduce server load

Cons.
1. limited storage.
2. Stale data (old data)
3. Security Concern.

Cache Architectures

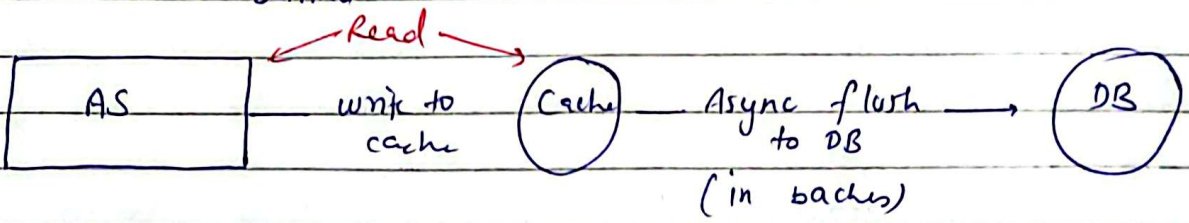
① Cache-Aside: cache only when need.
→ (1) external cache.

② Write-through. (less common to Cache-Aside)



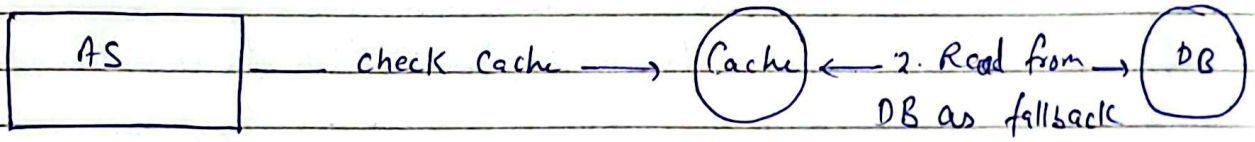
- Slow write
- burst to cache without need

③ Write-behind



- inconsistency If data lost in cache.

④ Read-through (similar to Cache-Aside)

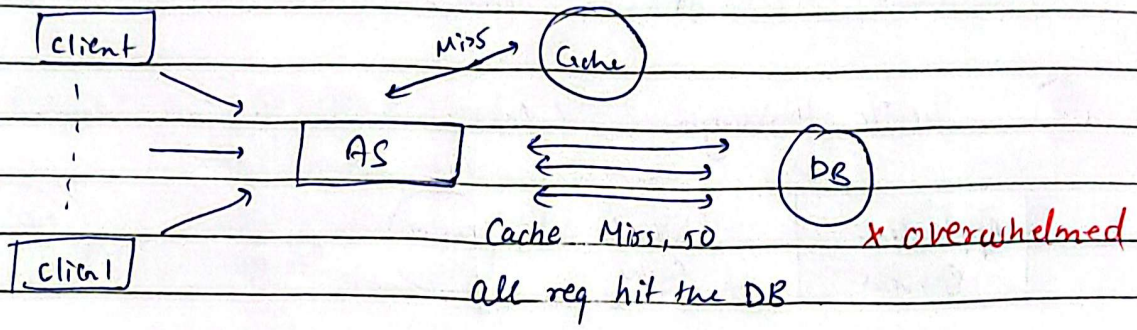


Cache Eviction Policies:

① LRU → most common and balanced default
② LFU → Good for highly skewed access patterns
③ FIFO → Simple, but rarely the right choice
④ TTL → Great for data that can go state, like API responses

Common Issues / Deep Dive.

① Cache Stampede (Thundering Herd Problem)



Problem → many clients/processes are waiting for the same resource/event, and when it becomes avail, all of them wake up and try to access it at the same time.

Solutions:

① for similar 10k req use only req to fetch the data from DB & cache it

② Distributed locking.

③ TTL = 60 sec + random (0, 10) = ex.

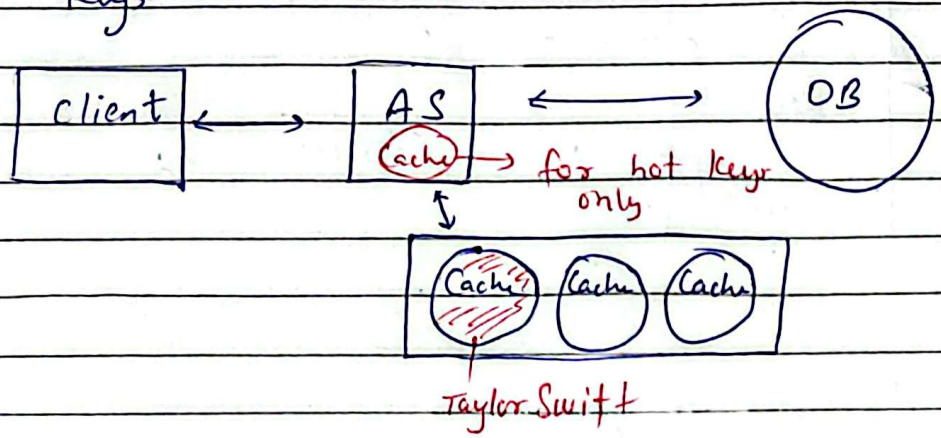
② Cache Consistency [still no perfect fix]

DB changed, cache not updated, users get STALE data.

Sol^{n} ① update cache when DB updated
② short TTL

③ Hot Keys

→ A hot key is a cache key that receives an unusually large no. of requests compared with other keys.



Soln ① maintain multiple copies of the hot value
② sharding : distribute ~~keys~~ req across multiple keys.
③ CDN → useful for hot, mostly-static content.
④ Rate limitting

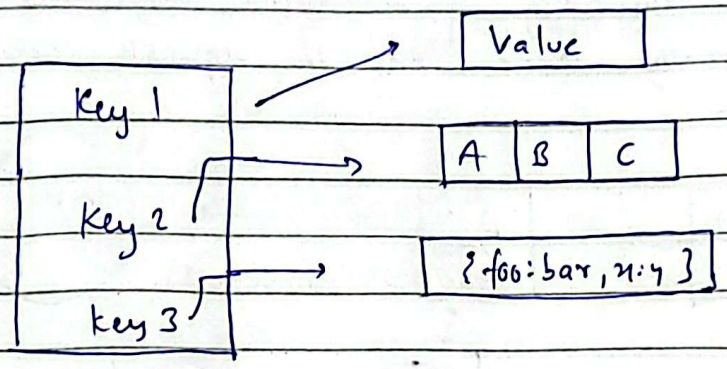
Caching in System Design Interviews

Q. When to Bring Up Caching:
- Ready - Heavy workload
- Expensive queries
- High database CPU
- Latency requirements

Q. How to Introduce Caching
① Identify the Bottlenecks.
② Decide what to Cache
③ Choose your cache architecture
④ Set an eviction policy
⑤ Address the downside.

CamScanner

Distributed Cache Design (Redis General)



Redis (key - value)

```
SET foo 1
GET foo      # Returns 1
INCR foo     # Returns 2
XADD mystream * name Sara surname OConnor
             # add an item
```

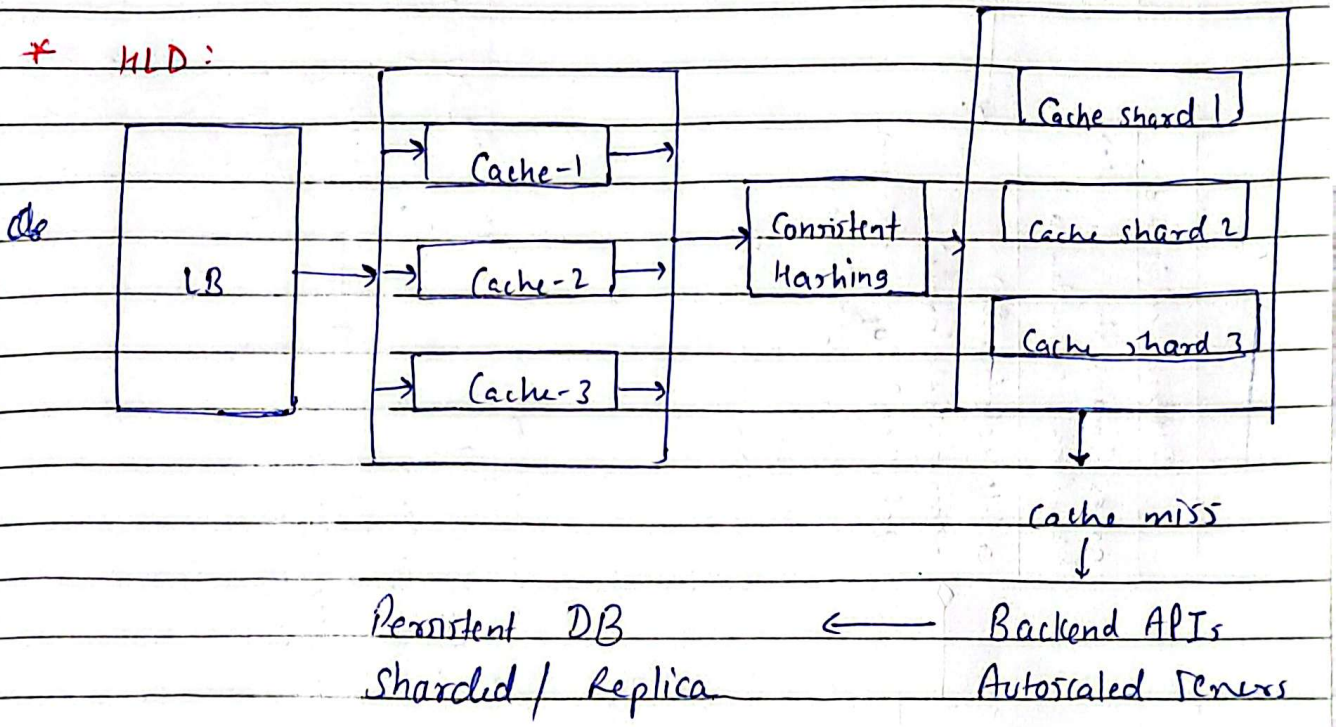
(1.1) Functional Requirements

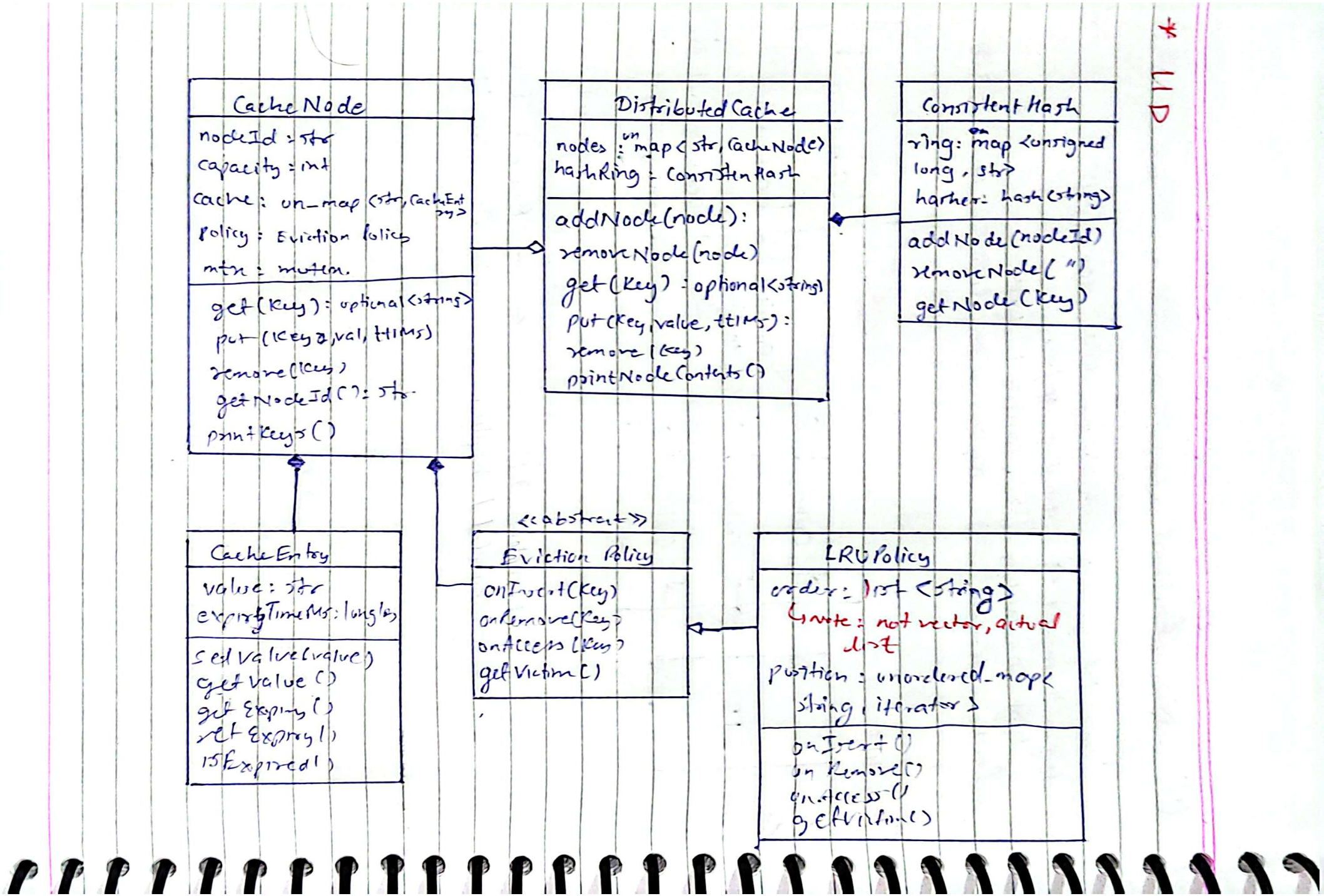
1. PUT : store a key-value pair with an optional TTL.
2. GET : return the value for a key or a miss.
3. DELETE: remove a key.
4. TTL expiry: expired keys are never returned.
5. Eviction : when memory is full, evict automatically using a configurable policy (LRU or LFU)
6. Elastic cluster: add or remove nodes at runtime with minimal data movement.
7. Replication and failover : data survives a node failure, and requests are routed around dead nodes.

(1.2) Non functional

Low latency → in-memory hash map O(1)
High throughput → 100 K+ op/sec
High availability → Consistent hash.
Fault Tolerance → Replicas on distinct nodes

* HLD:




CacheNode
nodeId : str
capacity : int
cache : un_map <str, CacheEntry>
policy : Eviction Policy
mtx : mutex.
get(Key) : optional<string>
put(key, val, ttlMs)
remove(key)
getNodeId() : str
printKeys()
DistributedCache
nodes : un map <str, CacheNode>
hashRing : ConsistentHash
addNode(node):
removeNode(node)
get(Key) : optional<string>
put(Key, value, ttlMs):
remove(Key)
printNodeContents()
ConsistentHash
ring: map <unsigned long, str>
hasher: hash<string>
addNode(nodeId)
removeNode("")
getNode(Key)
CacheEntry
value : str
expiryTimeMs : long long
setValue(value)
getValue()
getExpiry()
setExpiry()
isExpired()
<<abstract>>
Eviction Policy
onInsert(key)
onRemove(key)
onAccess(key)
getVictim()
LRUPolicy
order : list <string>
(note: not vector, actual list)
position : unordered_map <string, iterator>
onInsert()
onRemove()
onAccess()
getVictim()